

Ultrafast X-Ray Experiments

**Virtual Experiments at the X-Ray Laser European XFEL
Versuch EXP21**

Léonie Lanz

Nico Kalogerakis

September 16, 2026

Contents

1. Introduction	1
2. Theoretical Background	1
2.1. Synchrotronradiation	1
2.1.1. Free Electron Laser (FEL)	1
2.2. Pump-probe-spectroscopy	2
2.2.1. The pump-pulse	2
2.2.2. The probe-pulse	2
2.2.3. Transient difference spectrum	2
2.3. The investigated molecule	3
2.3.1. Electronic Structure of $[\text{Fe}(\text{bipy})_3]^{2+}$	3
2.3.2. Relaxation process	4
3. Experimental setup	4
3.1. Beamline	4
3.2. Focusing optics	5
3.3. Sample environment	6
3.4. Diagnostic table	6
4. Experimental Procedure	6
4.1. Preparation of the Experiment	6
4.1.1. Alignment and Characterization of the X-ray Beam	6
4.1.2. Preparatory Estimates for the Experiment	9
4.2. Performing the Experiment with Data Acquisition	11
5. Analysis and Interpretation	13
5.1. XES spectra at 100ps	13
5.2. Difference spectra for different pump-probe delays	14
6. Discussion of the physical meaning	17
6.1. Error analysis	18
7. Conclusion and outlook	18
References	A
Abbildungsverzeichnis	B
Appendix A. Additional Plots	C
Appendix B. Code	C

1. Introduction

The experiment EXP21 - Ultrafast X-ray experiments is there to gain basic hands-on knowledge about the methods of ultrafast pump-probe spectroscopy as well as X-ray emission spectroscopy and related X-ray techniques. This is done through the usage of the virtual environment VLab - a software that simulates a beamline of the European XFEL. The following procedures are part of the experiment: characterising the optics of the X-ray beam used, making the preparatory estimates and calculations surrounding the setup of the experiment and finally executing a pump-probe measurement of the relaxation process of the iron-compound $[\text{Fe}(\text{bipy})_3]^{2+}$ at different time delays.

2. Theoretical Background

2.1. Synchrotron radiation

When accelerating electrons or other charged particles to relativistic speeds along a curved trajectory they emit so-called synchrotron radiation. It gains its name from the ring accelerators, the synchrotrons, where it was first observed. A typical synchrotron facility consists of a linear accelerator leading into a booster ring in which the electrons are brought up to relativistic speeds [1][2]. The electrons are then fed into a storage ring in which they continue to circulate at relativistic speeds. From the storage ring the beam-lines lead tangentially away in which the synchrotron radiation is utilized in experimental hutches [1][2].

Synchrotron radiation is bright X-radiation with a wide tunability in energy, which is useful for examining structures and dynamics of materials, as X-ray photons have energies in the order of several kiloelectronvolts (keV) [2]. This means their wavelengths are in the order of magnitude of Angstroms and comparable to the spacing of atoms in solid materials. This is useful, as it means that atoms in a crystalline array can diffract X-ray beams and the interference patterns can be measured [1].

There are four generations of synchrotrons with the main difference being the brilliance achieved [1]. The first was the bending magnet in the synchrotron as it was used for high energy collider physics with no additional modifications. The second generation introduced the wiggler, a configuration of dipole magnets with alternating polarity to achieve synchrotron radiation a linear path. The third generation synchrotron radiation facilities were designed for low emittance of the beams with more effective linear magnet configurations, the undulators [2]. A low emittance means a smaller angular divergence of the beam and higher brilliance [2].

2.1.1. Free Electron Laser (FEL)

Free-electron lasers are coherent radiation sources that are capable of even higher brilliance than third generation synchrotron facilities [3]. They are, in theory, able to operate at any arbitrary wavelength or spectrum of wavelengths with the limit being the energy and quality of the accelerated electron beam [3]. FELs use a process called self-amplified stimulated emission (SASE). SASE free-electron lasers typically use monochromatic electron beams with a long, high-quality undulator around the order of magnitude of a few hundred metres. The light waves emitted by the electron bunches will line up in phase and amplify themselves through superposition. As the electrons oscillate down the undulator they are also interacting with the magnetic components of the light they themselves are generating. The transverse component of the electrons oscillation and the magnetic field component of the emitted light create a Lorentz force which acts upon the electrons and moves them along axial directions. This leads to the electrons creating microbunches inside their conventional bunches [1].

The duration of a single X-ray pulse can also be reduced to even less than a few femtoseconds. This allows for an investigation of molecular processes and photochemical reactions through a form of sequential snapshots with a characteristic time of under a picosecond [1].

2.2. Pump-probe-spectroscopy

The principle of pump-probe-spectroscopy is sending two lasers at the same object with a certain time delay so that the first laser, an optical laser, can excite the system in some way and the second laser, an X-ray pulse can measure the response (see Figure 1). In a FEL such as the European XFEL this can be done with ultrafast X-ray pulses on a timescale of a few femtoseconds (10^{-15} s). Varying the time delay allows for a series of sequential snapshots of the process observed and varying the wavelength of the excitation laser allows for an investigation of different processes [4].

The selection of the different discrete time delays is important. For negative time delays ($\Delta t < 0$) the probe pulse arrives before the pump pulse and thus measures just the ground state and for a delay of $\Delta t = 0$ both pulses overlap exactly. Whereas for positive delays ($\Delta t > 0$) the probe pulse measures the developing dynamics.

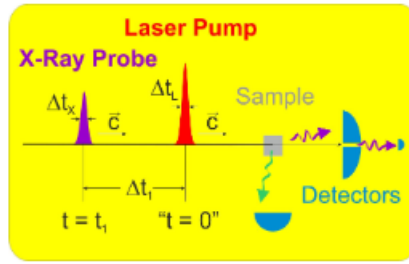


Figure 1: A schematic of a pump-probe-experiment [4].

2.2.1. The pump-pulse

The pump pulse acts as a kick-off signal for the process. It excites the molecule at a well-defined time ($t = 0$) from its thermal ground state into a higher energetic state [4].

In this experiment specifically the pump pulse is an ultra-short optical laser-pulse with a wavelength in the VIS/UV range which excites the iron-compound $[\text{Fe}(\text{bipy})_3]^{2+}$ from its singlet low-spin-state (1A_1) into the excited state $^1\text{MLCT}$ (metal-to-ligand charge transfer) after which the relaxation process starts [4].

2.2.2. The probe-pulse

The probe pulse is a second pulse that hits the probe with a known time delay to 'photograph' the current state. In this experiment an ultra-short X-ray pulse from the European XFEL is used. It ionises the 1s core electron of the central iron atom. The resulting X-ray emission ($K\beta$ -XES) is measured which is sensitive to spin- and structure changes of the iron [4].

2.2.3. Transient difference spectrum

Most of the time during the experiment, only a fraction of the molecules in the probe are in an excited state. This means the pure excited spectrum is hard to see directly. To get a useful reading of the data the transient signal in the difference spectrum is used:

$$\text{transient signal} = I_{\text{laser on}}(\Delta t) - I_{\text{laser off}} \quad (1)$$

The transient signal shows the physical changes the best: Negative parts of the signal mean a depopulation - places in the spectrum where the intensity lessens as the ground state is emptied because of the laser, positive parts are new spectral features such as the growing of a spin-sensitive satellite peak which indicates the existence of intermediate states.

When plotting the intensity of these notable energies against the time delays, you get so-called kinetic traces from which the lifespans of the intermediate states is able to be determined.

2.3. The investigated molecule

We investigated a molecule formed of an Iron center (Fe^{2+}), a transition metal ion, and 3 attached ligands (bipyridine). This forms a chemical complex compound. The central atom acts as an electron acceptor, while the ligands act as the electron donor.

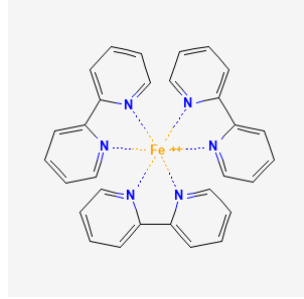


Figure 2: Schematic view of $[\text{Fe}(\text{bipy})_3]^{2+}$

This bonding can take place because of the d^6 electron configuration of Fe^{2+} and the nitrogen atoms in the pyridine rings. In our case we have two bridged pyridine rings that form the uncharged bidentate ligand bipyridine (abbreviated bipy). Both nitrogen atoms in each of the 3 ligands bind to the central Fe^{2+} atom.

2.3.1. Electronic Structure of $[\text{Fe}(\text{bipy})_3]^{2+}$

Because of the electrostatic interaction between the ligands and the d electrons of the Fe^{2+} , the 5 d orbitals, that normally would be energetically equivalent, with a maximised spin multiplicity, are now energetically raised or lowered due to the nature of interaction with the ligand field (see Figure 3). The energy of d orbitals pointing to the ligands (e_g) is raised, while the energy of those pointing between the ligands is lowered (t_{2g}). The difference of energy Δ_{oct} is called the strength of the crystal field.

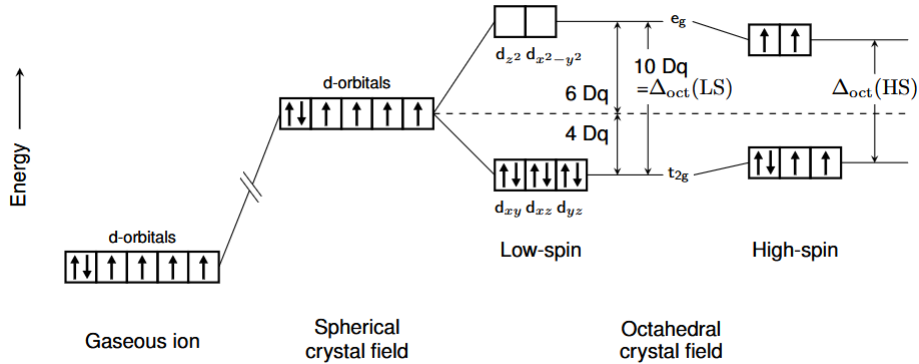


Figure 3: Energy level diagram of the d orbitals of a d^6 transition metal ion in an octahedral crystal field. An assumed spherical ligand field raises the energy of the d orbitals uniformly by approximately 20 eV to 40 eV. The degeneracy of the d orbitals is removed by the actual octahedral field of the point-charged ligands. [4]

Following Hund's rule the lower-energy orbitals (t_{2g}) are filled first (one by one with electrons of the same spin). In the low-spin state our d^6 metal ion has no unpaired electrons (total Spin $S = 0$). That represents the singlet state 1A_1 . In the high spin state it has the maximum of unpaired electrons ($S = 2$). We have a quintet state 5T_2 . The energy difference Δ_{oct} is smaller for HS configs, because the e_g level is filled and those electrons can push the ligand atoms further away that in LS configs. Excitation to the HS state occurs in this experimental setup through photoexcitation to higher lying MLCT bands followed by relaxation.

2.3.2. Relaxation process

The actual process we want to look at is the relaxation process that occurs after exciting the iron compound from the singlet state 1A_1 into the MLCT (metal to ligand charge transfer) band. In the electronic ground state $[\text{Fe}(\text{bipy})_3]^{2+}$ is a low spin complex while it's a high spin complex when excited to the MLCT band.

There are two different ways the deactivation could occur. Either directly to the high spin state $^5T_{2g}$ or indirectly via the transient state $^3T_{1g}$. Both processes are equally possible. The average lifetime of the MLCT band is $\tau_{MLCT} \approx 100$ fs while the average lifetimes of the transient state $^3T_{1g}$ and the high spin state $^5T_{2g}$ are $\tau_3 < 100$ fs and $\tau_5 \approx 1$ ns respectively (see Figure 4) [4].

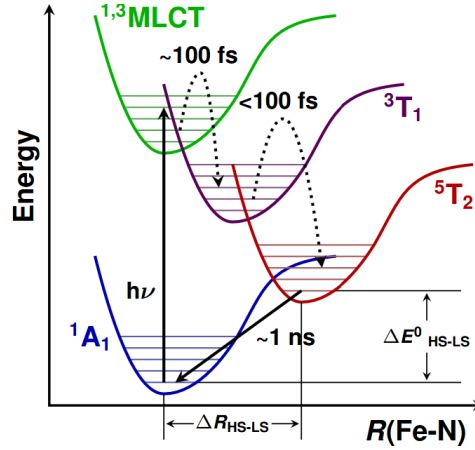


Figure 4: The potential energy curves of the excited state dynamics of $[\text{Fe}(\text{bipy})_3]^{2+}$ in H_2O after the excitation to the MLCT band. [4].

3. Experimental setup

The entire experiment is conducted in VLab, a software modeled after a hutch in the experimental hall of the European XFEL in Schenefeld. The European XFEL is 3.4 km long and runs from the DESY research center to the European XFEL campus. A 1.9 km long superconducting linear accelerator feeds electrons with energies up to 17.5 GeV into a branched tunnel system with two 212 m long undulators that generate hard X-radiation in the range of 5 keV to 25 keV and one 125 m long undulator that generates soft X-radiation in the range of 0.25 keV to 3 keV.

VLab recreates the Femtosecond X-ray Experiment research platform FXE. The simulation has no built-in structural errors, but there are statistical errors when it comes to the photon count.

3.1. Beamline

After the SASE process in the undulators, the beam is neither monochromatic nor really polychromatic - it's a so-called "pink" beam - which is why it goes through a monochromator before entering any of the optical elements on the optical bench in the experimental hutch.

The beamline in VLab uses a four-crystal monochromator (see Figure 5) which uses the diffraction in the first crystal to absorb all wavelengths that do not interfere constructively [4]. The four crystal setup ensures that the monochromatic beam follows the same path after the monochromator the pink beam would have.

Before entering the optical bench there's also a beam splitter built with a linear diamond grating which can be inserted, if so desired, to look at the ± 1 order diffractions. The beam splitter is built in such a way, that one of the first order diffractions is sent to a spectrum analyser which consists of a bent crystal and a detector [4].

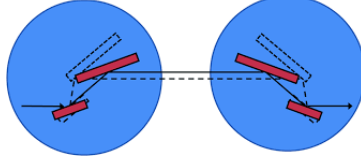


Figure 5: Optical layout of a four-crystal monochromator [5].

3.2. Focusing optics

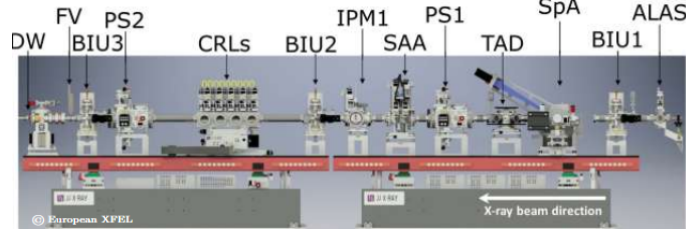


Figure 6: The optical bench in the FXE hut with the following components implemented: BIU 1,2,3 (beam imaging units), SpA (spectrum analyser), TAD (time arrival detector), PS 1,2 (power slits), SAA (solid attenuator assembly), IPM1 (intensity position monitor), CRLs (compound refractive lenses) [4].

In VLab as well as in real life, there are multiple optical elements for controlling and characterising the beam (see Figure 6). Listed here are the ones we actively used during this experiment.

There are three BIUs which measure the intensity distribution of the beam perpendicular to its propagation direction. In VLab these intensity distributions can be saved as files with a size of 420x420 pixel and the screen dimension 10 mm x10 mm leading to a resolution of 0.024 mm/px.

The power slits are there to restrict the areas of the beam, for this experiment they are fully opened.

As some components and detectors of the beamline will be damaged when hit with the full force of the X-ray beam (such as the BIUs), it is necessary to have a way of weakening the beam without changing anything else about its optics. This is done with the solid attenuators of the SAA which are able to weaken the beam by multiple orders of magnitude. It consists of an assembly of four arms (or rods) which each have a selection of foils of varying thicknesses to hold into the beam. The foils are made of diamond (C*), silicon (Si) and boron carbide (B₄C) [4].

To focus the beam, compound refractive lenses made of beryllium are used. X-ray optics need a different approach to lenses than UV/VIS light as can be seen when looking at the dimensionless wavelength dependent index of refraction: $n(\lambda) = n_R(\lambda) + in_I(\lambda)$. The real part n_R is the refractive index and measures the phase velocity of light. For wavelengths in the X-ray regime it drops to less than unity, which means the refractive power is extremely low. This means focusing of the beam has to be performed with concave lenses rather than with the optical lenses one would use to focus an optical beam. The lenses are made of beryllium as it is one of the available materials with a low atomic number which means that the lenses have low X-ray absorption [4].

The focal distance F of single refractive X-ray lenses is dependent on the lens radius R and the refractive decrement δ through $F = \frac{R}{2\delta}$ but as δ is very low it's necessary to use stacks of several lenses. Now the focal distance of N lens elements is given by:

$$F = \frac{R}{2N\delta} \quad (2)$$

The CRLs in VLab are in the form of ten retractable carrier cassettes which are each loaded with up to ten individual Be lenses with varying R (see Table 2). The carrier cassettes can be placed into the beam at fixed positions which are already aligned to perfectly focus the beam - saving the work of manually finding the right positions that would be required when actually working at the FXE [4].

3.3. Sample environment

The next section of the hutch modeled in VLab is the sample environment in which the actual experiment takes place. It consists of the sample table on which the pump-laser and the X-ray hit the probe and secondary spectrometers as well as the large pixel detector (LPD). For this experiment the liquid beam injector on the sample table is used to make sure that the probe for every pump and probe cycle is fresh [4].

There are two secondary spectrometers available in VLab: the Von Hamos spectrometer and the Johann spectrometer. The Von Hamos is a dispersive spectrometer which means that it is better suited for recording an entire spectrum while the Johann is a monochromatic spectrometer and better suited for accurately measuring the wavelength of single small wavelength ranges of interest. The Von Hamos in VLab consists of up to 16 cylindrically curved crystals [4].

The output of the spectrometers can be measured by two different detectors: the Gotthard detector which is a MHz compatible strip detector or the Jungfrau detector which is a 10 Hz compatible pixel detector. These detectors are made out of silicon and work because of the X-ray generating conduction band electrons whose charge quantity is then measured. This means the detectors sensitivity is energy-dependent.

The LPD was developed specifically for the European XFEL. It's an 1 mega pixel detector with the ability to resolve single pulses within a pulse train. To ensure that the directly transmitted X-ray beam doesn't destroy any of the sensors in the detector there's a small hole which allows the beam to pass without directly hitting any of them.

In this section there's also another more precise BIU - the X-ray eye - which also measures intensity distributions with a size of 420x420 pixel and the screen dimension 2 mm x 2 mm leading to a resolution of 0.005 mm/px.

3.4. Diagnostic table

The relevant part of this last section of the beamline is the final beam stop, made of a sequential set of B4C and Cu attenuators. The failsafe is a "burn-through" monitor placed in front of the beam stop materials. If the beam is accidentally focussed to the end of the hutch, it will burn a hole in this monitor and generate visible light which switches off the electron photoinjector.

In VLab the worst thing that can happen in regards to radiation safety is that you burn through one of your detector screens and have to start over again which is of course much less punishing than a potentially life threatening scenario in real life.

4. Experimental Procedure

4.1. Preparation of the Experiment

4.1.1. Alignment and Characterization of the X-ray Beam

Before doing anything on a beam line it's important to characterize it.

As the absorption edge for the Fe $K\beta_{1,3}$ line of 7065 eV, the energy of the xray beam is set to 7400 eV, to avoid the ripples near the reaction (as illustrated by the X-ray absorption spectrum of $[\text{Fe}(\text{terpy})_2]^{2+}$ Figure 7) but without too strong of a loss in intensity.

To measure the initial X-ray beam the power slits are fully opened and the following attenuator foils are used to bring the beam to an intensity just below saturation of the beam imaging unit 2 (BIU2):

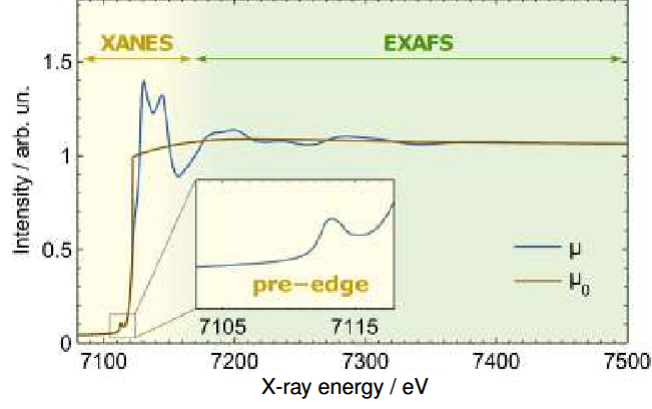


Figure 7: X-Ray absorption spectrum of $[\text{Fe}(\text{terpy})_2]^{2+}$ (terpy: terpyridin) at the Fe K edge displaying the XANES and EXAFS regions including the background absorption contributions μ_0 . The inset zooms into the weak pre edge features [4].

Table 1: Attenuator foils used

Rod	Attenuator foil
1	C 0.1
2	Si 0.05
3	C 0.4
4	B4C 0.8

The size and position of the x-ray is now measured using BIU2. Then the four-bounce crystal-monochromator is turned on and then off again to record the reduction of the spectral bandwidth. Then the dispersive single-shot spectrum analyser is used to record the spectrum of the pink X-ray beam with a bragg angle of 24° .

For the following experiment the pink X-ray beam is used even though the monochromatic has a higher resolution, as the intensity of the beam suffers about three orders of magnitude under the monochromator (see Figure 8 and).

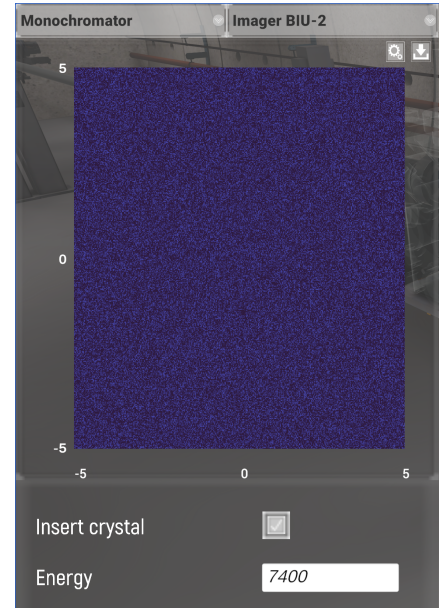
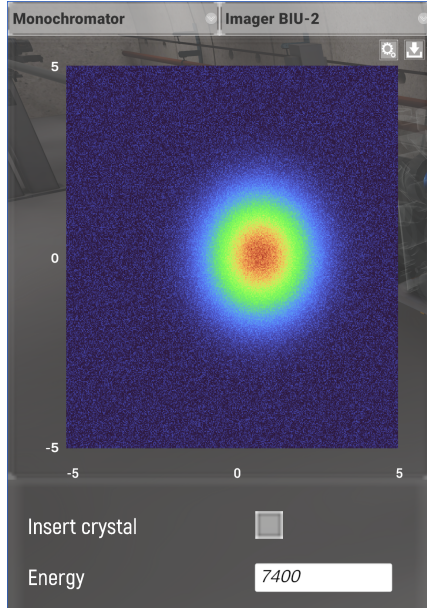


Figure 8: To the left is the attenuated pink beam, to the right the monochromatic beam with the same attenuators in.

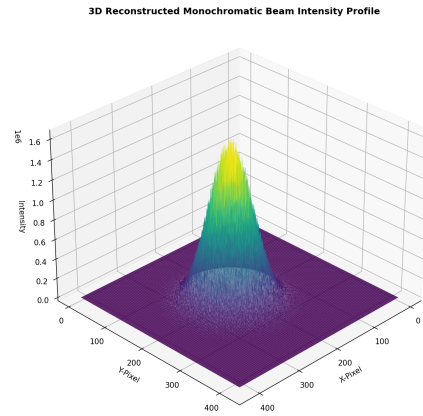
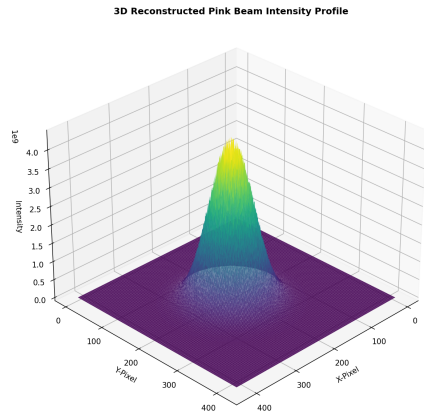


Figure 9: To the left is a 3D intensity plot of the pink beam, to the right the 3D intensity plot of the monochromatic beam with the same attenuators in.

The last step of diagnosing the beam line is using the refractive beryllium X-ray lenses to focus the beam at the imaging unit X-ray eye. It's important to appropriately adjust the attenuator foils as to not burn the imaging unit with a focussed beam.

Table 2: A list of the beryllium lenses available in VLab together with the arms they were mounted on (arm 10 the most downstream), the Number of lenses in each cassette N and the central curvature of the lenses R [4].

Arm	10	9	8	7	6	5	4	3	2	1
N	10	10	8	7	5	9	4	10	6	1
R/mm	0.5	0.5	0.5	0.5	0.5	1.0	0.5	1.5	1.5	0.5

With beryllium lenses 1, 2 and 5 (taken from Table 2) and appropriate attenuator foils inserted, a beam with the FWHM of $\approx 13 \mu\text{m}$ (siehe Figure 10) is achieved. The total focus length is calculated with

$\delta \approx 7 \cdot 10^{-6}$ [4] and the properties of the lenses themselves taken from Table 2:

$$F = \frac{R}{2\delta N} \Leftrightarrow \frac{1}{F_{ges}} = \frac{2\delta N_1}{R_1} + \frac{2\delta N_2}{R_2} + \frac{2\delta N_5}{R_5}$$

$$\Leftrightarrow F_{ges} = \frac{1}{2\delta(\frac{N_1}{R_1} + \frac{N_2}{R_2} + \frac{N_5}{R_5})} \approx 4.76 \text{ m}$$

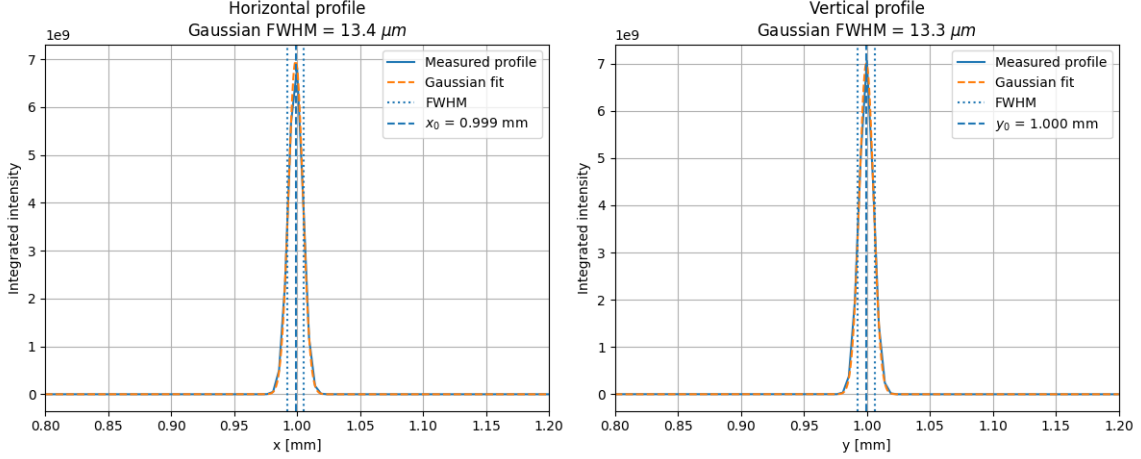


Figure 10: The horizontal and vertical profile of the focussed X-ray beam plotted together with a gaussian fit and the corresponding FWHM of $13.5 \mu\text{m}$.

4.1.2. Preparatory Estimates for the Experiment

The transition metal complex to be studied, $[\text{Fe}(\text{bipy})_3]^{2+}$, is obtained through dissolving the salt $[\text{Fe}(\text{bipy})_3]\text{Cl}_2$ in deionized water to the desired concentration. As the actual sample is a powder consisting of small microcrystals that contain incorporated water which remained in the crystal lattice after the crystallization process.

A minimum volume of 50 ml is required to operate the liquid jet in the experiment. To ensure that the sample is completely refreshed between probe pulses, the jet needs to have a certain flow speed with an assumed repetition rate of the X-ray pulse train of 1.1 MHz and an assumed top hat beam profile of the X-ray laser with the dimension $10 \mu\text{m} \times 10 \mu\text{m}$. The minimum required flowspeed is calculated as follows:

$$\Delta x_{\text{travel}} \geq w \quad \Rightarrow \quad v_{\text{min}} \cdot \Delta\tau \geq w \quad \Rightarrow \quad v_{\text{min}} \geq \frac{w}{\Delta\tau} \Rightarrow v_{\text{min}} = 11 \text{ m s}^{-1}$$

With $w = 10 \mu\text{m}$ being the spatial width and $\Delta\tau = \frac{1}{f} = \frac{1}{1.1 \times 10^6 \text{ Hz}} \approx 9.09 \times 10^{-7} \text{ s}$ the characteristic time.

There are three laser wavelengths available for excitation: 355 nm, 400 nm and 515 nm. To evaluate which fits the experiment best, the concentration c of the solution for the three wavelengths for a desired optical density of $OD = 1$ and a liquid jet thickness of $d = 25 \mu\text{m}$. The optical density is defined as $OD = \epsilon \cdot c \cdot d$ with $\epsilon(\lambda)$ the wavelength dependent extinction coefficient of $[\text{Fe}(\text{bipy})_3]^{2+}$ in water. The extinction coefficients can be estimated to be $\epsilon(355 \text{ nm}) = 12000 \text{ M}^{-1} \text{ cm}^{-1}$, $\epsilon(400 \text{ nm}) = 4000 \text{ M}^{-1} \text{ cm}^{-1}$, $\epsilon(515 \text{ nm}) = 8000 \text{ M}^{-1} \text{ cm}^{-1}$ as taken from Figure 11.

The following concentrations were calculated:

$$c_{355} = 0.033 \text{ mol/L}, c_{400} = 0.100 \text{ mol/L}, c_{515} = 0.050 \text{ mol/L}$$

There are different pulse energies available for each excitation wavelength: $5 \mu\text{J}(355 \text{ nm})$, $15 \mu\text{J}(400 \text{ nm})$ and $50 \mu\text{J}(515 \text{ nm})$. With that in mind the wavelength of 400 nm was chosen, as it has a higher pulse energy, even though it's not ideal when looking at the low absorption rate. The extinction coefficient of

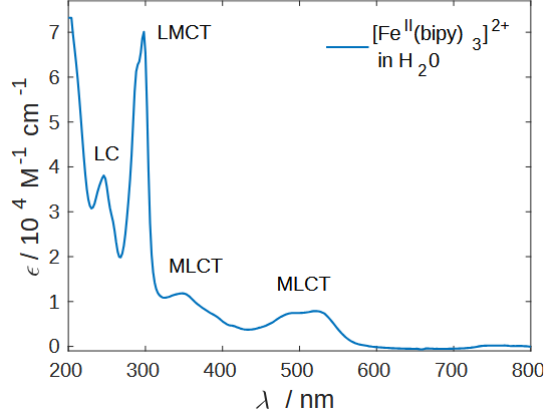


Figure 11: UV/VIS absorption spectrum of an aqueous $[\text{Fe}(\text{bipy})_3]\text{Cl}_2$ solution. The different CT transitions are indicated. The measured molar absorptivity ϵ values lie in the expected range of 10^{-4} $\text{l mol}^{-1} \text{cm}^{-1}$ to 10^{-5} $\text{l mol}^{-1} \text{cm}^{-1}$ for dipole-allowed transitions[4]

the 515 nm is slightly better but with the low pulse energy it will be hard to see anything.

As we want to ensure that the X-ray beam always hits excited molecules we need to determine the diameter of the optical laser beam as well as the diameter of the X-ray beam correctly in relation to each other.

The optical laser beam profile is approximately gaussian-shaped and has an original diameter of 10 mm (FWHM). This collimated beam is focussed onto the liquid jet by a lens with a focal length of $f = 250$ mm. From the Rayleigh criterion $d_{\text{Airy}} = 2.44 \cdot \frac{\lambda \cdot f}{D}$ we calculate $d_{\text{Airy}} = 24.4 \mu\text{m}$ or rather $d_{\text{FWHM}} = ?$ with our chosen wavelength of 400 nm.

If the optical laser and the X-ray laser had the same diameter, the x-ray would also investigate molecules that were excited only weakly or even not at all, as the optical laser has an approximately Gaussian beam shape. Ideally, the X-ray is positioned exactly in the centre of the ca. 10 times bigger optical laser beam. Then it can investigate a maximal volume of excited molecules. This also compensates for the fact that the optical laser jitters in the focus layer of the X-ray due to not being focussed there itself.

These estimates have consequences for our needed flow speed of the liquid jet which has to be 10 times faster to adjust to the new diameter of the optical laser.

Ideally, the number of absorbed optical laser photons should be less than or approximately equal to the number of sample molecules in the laser-excited volume. This reduces the likelihood that multiple photons hit the same molecule and excite it into states other than the one we want to observe while still optimizing the number of excited molecules.

To ensure the optical laser excites the sample efficiently, the laser intensity ideally is in a range where the number of optical laser photons is less than or approximately equal to the number of sample molecules in the laser-excited volume.

First we calculate the number of $[\text{Fe}(\text{bipy})_3]^{2+}$ molecules in the volume illuminated by the laser with a chosen beam size of $d_L = 150 \mu\text{m}$, a concentration of $c = 1.00 \times 10^{-4} \text{ mol/cm}^3$, a wavelength of $\lambda = 400$ nm and a jet size of $d_{\text{jet}} = 25 \mu\text{m}$. The particle density is calculated by:

$$\mathcal{N} = c \cdot N_A = 0.100 \text{ mol/L} \cdot 6.022 \times 10^{23} \text{ L}^{-1} \text{ mol}^{-1} = 6.022 \times 10^{22} \text{ molecules/cm}^3 \quad (3)$$

The illuminated cross-section area A of the gaussian beam is $A_{\text{eff}} = 1.133 \cdot d_{\text{FWHM}}^2 = 1.133 \cdot (0.015 \text{ cm})^2 \approx 2.549 \times 10^{-4} \text{ cm}^2$ from which the effective volume can be calculated: $V_{\text{eff}} = A_{\text{eff}} \cdot d_{\text{jet}} = 2.549 \times 10^{-4} \text{ cm}^2 \cdot 25 \times 10^{-4} \text{ cm} = 6.373 \times 10^{-10} \text{ L}$. The number of molecules is then calculated:

$$N_{\text{molecules, eff}} = \mathcal{N} \cdot V_{\text{eff}} = (6.022 \times 10^{22} \text{ cm}^{-3}) \cdot (6.373 \times 10^{-7} \text{ cm}^3) \approx 3.838 \times 10^{16} \text{ molecules} \quad (4)$$

Now to determine the pulse energy at which the number of photons per pulse is equal to the number of molecules just calculated:

$$N_{\text{photons}} = N_{\text{molecules}} \quad (5)$$

$$E_{\text{photon}} = \frac{h \cdot c}{\lambda} = \frac{(6,626 \times 10^{-34} \text{ J} \cdot \text{s}) \cdot (2,998 \times 10^8 \text{ m/s})}{400 \times 10^{-9} \text{ m}} \approx 4.966 \times 10^{-19} \text{ J} \quad (6)$$

$$E_{\text{pulse}} = N_{\text{molecules}} \cdot E_{\text{photon}} = (3.838 \times 10^{16}) \cdot (4.966 \times 10^{-19} \text{ J}) \approx 19.06 \times 10^{-3} \text{ J} \quad (7)$$

For a wavelength of 400 nm there's a maximal pulse energy of 19 059.51 mJ available for a spot diameter of 150 μm

There are two X-ray crystal spectrometers available to use: a von Hamos spectrometer and a Johann spectrometer. As the von Hamos spectrometer can record the entire spectrum at once while also being immune to intensity variation, in contrast to the Johann spectrometer which needs to scan each wavelength separately and requires normalization to the accumulated incident intensity at each measurement point to measure the whole spectrum, it is better suited for this experiment.

In order to choose the correct crystal for the emission spectrometer the Bragg reflection angle needs to be considered which means we need to calculate the Bragg reflection angle for the Fe $K\beta_{1,3}$ emission line. From the FXE website [?] we preselect the Si(531) crystal and the Ge(531) crystal. The Bragg angle is calculated through:

$$\lambda = 2d_{hkl} \cdot \sin(\theta_B) \quad (8)$$

with $\lambda_m = 1.7566 \text{ \AA}$ for the Fe $K\beta_{1,3}$ main line ($E_m = 7058 \text{ eV}$), $\lambda_s = 1.7596 \text{ \AA}$ for the Fe $K\beta'$ satellite line ($E_s = 7046 \text{ eV}$), d calculated through the crystal planes (hkl) and the lattice constants $a_{\text{Si}} \approx 5.431 \text{ \AA}$ & $a_{\text{Ge}} \approx 5.658 \text{ \AA}$ to $d_{\text{Si}} = \frac{5.431}{\sqrt{35}} \approx 0.918 \text{ \AA}$ and $d_{\text{Ge}} = \frac{5.658}{\sqrt{35}} \approx 0.956 \text{ \AA}$.

For the Fe $K\beta_{1,3}$ main emission line the following Bragg angles were calculated:

$$\theta_{B_{\text{Si}}} = \arcsin\left(\frac{1.7566}{2 \cdot 0.9180}\right) \approx 73.09^\circ \quad (9)$$

$$\theta_{B_{\text{Ge}}} = \arcsin\left(\frac{1.7566}{2 \cdot 0.9564}\right) \approx 66.69^\circ \quad (10)$$

For the Fe $K\beta'$ satellite line the following Bragg angles were calculated:

$$\theta_{B_{\text{Si}}} = \arcsin\left(\frac{1.7596}{2 \cdot 0.9180}\right) \approx 73.42^\circ \quad (11)$$

$$\theta_{B_{\text{Ge}}} = \arcsin\left(\frac{1.7596}{2 \cdot 0.9564}\right) \approx 66.92^\circ \quad (12)$$

For a Bragg angle that is closer to 90° the spectral resolution is better. As Si(531) has a Bragg angle of 73.1° which is the highest of the two materials, it was chosen for the Von Hamos even though germanium has a higher atomic number than silicium and would have a higher reflection-efficiency.

4.2. Performing the Experiment with Data Acquisition

Using the Large Pixel Detector one can optimize the position of the liquid jet in the X-ray beam for maximal intensity of elastic scattering. The maximal intensity achieved with a pink beam was $3.8 \cdot 10^{15}$. The X-ray emission spectrometer was adjusted by setting the Bragg angle range to 74.49° to 72.03° on the crystal and 70.43° to 75.53° on the detector and orienting the detector perpendicular to the incoming dispersive spectrum (see Figure 12).

The excitation laser was set to $\lambda = 400 \text{ nm}$ and $f = 0.02$ with a beam diameter of $d = 150 \mu\text{m}$ in the sample plane. The spatial overlap between laser and X-ray beam was adjusted so that the X-ray laser always overlapped with the optical laser even with the jitter of the optical laser. The corresponding beam

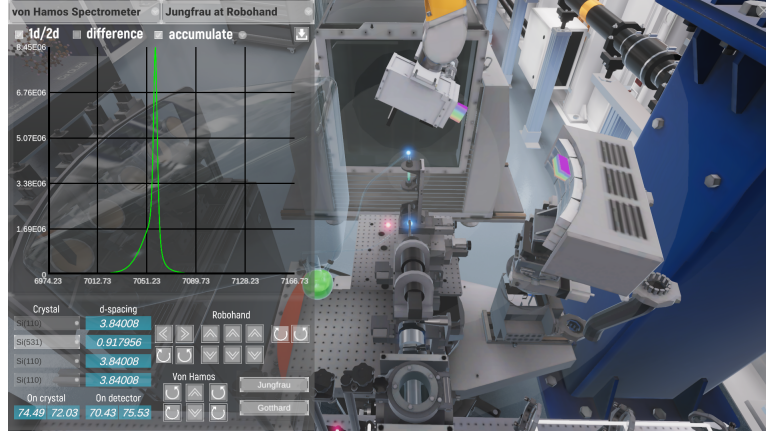


Figure 12: The Von Hamos spectrometer in operating position. with the read-out of the Jungfrau detector as well as the settings for the position to the left.

profiles can be seen in Figure 15, while the laser beam profile can be seen in Figure 14 and the X-ray beam profile in Figure 13.

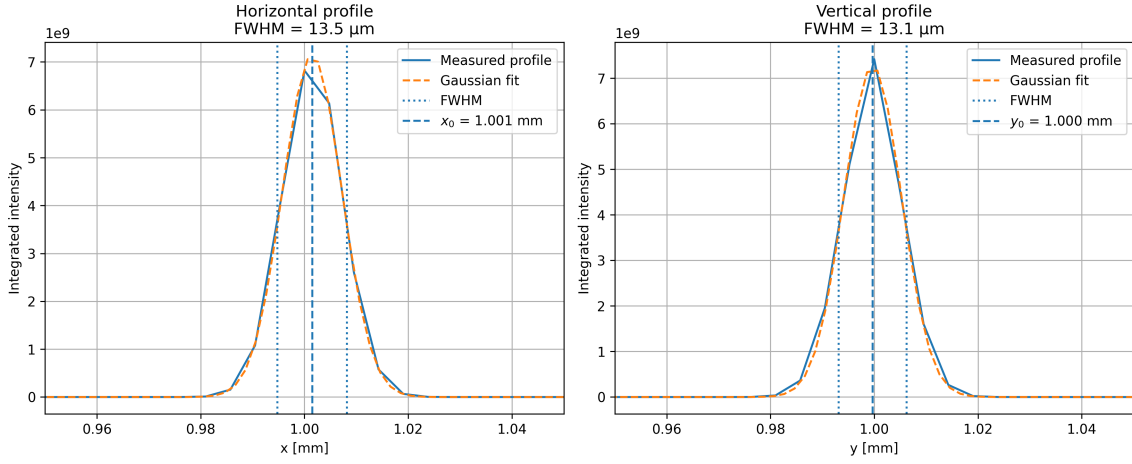


Figure 13: The profile of the X-ray beam measured in the sample plane.

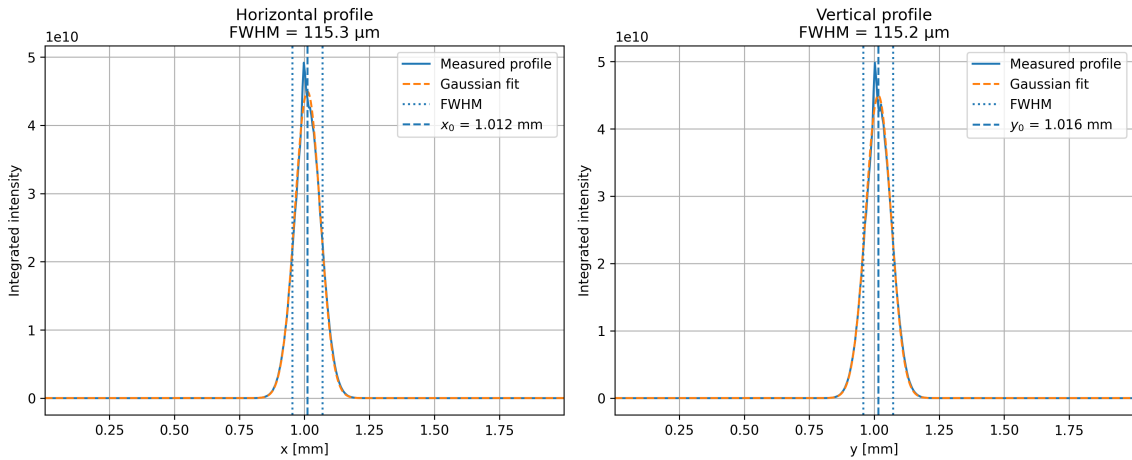


Figure 14: The profile of the laser beam measured in the sample plane.

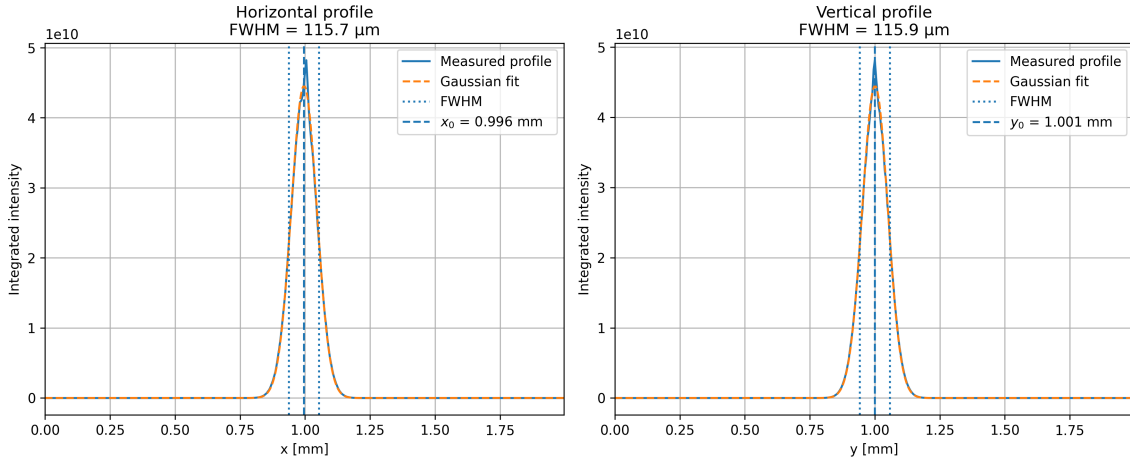


Figure 15: The profile of the overlapping laser and X-ray beams in the sample plane.

First the time delay of the laser was set to 100 ps. This time delay means the laser pulse definitely strikes the sample before the X-ray pulse arrives. XES spectra with the laser on and off were recorded, as well as a difference spectrum.

For the main measurement of the photophysical/photochemical reaction of $[\text{Fe}(\text{bipy})_3]^{2+}$ the range of delays was chosen to be -150 fs to 800 fs with a step size of 50 fs between -150 fs and 400 fs and a step size of 200 fs between 400 fs and 800 fs. For each delay ten difference spectra were recorded with an accumulation time of $60 \text{ s} \pm 2 \text{ s}$ each, which was tracked using a stopwatch.

5. Analysis and Interpretation

The analysis of the measured data is carried out using provided reference spectra resolved into the different spin states (see Figure 16).

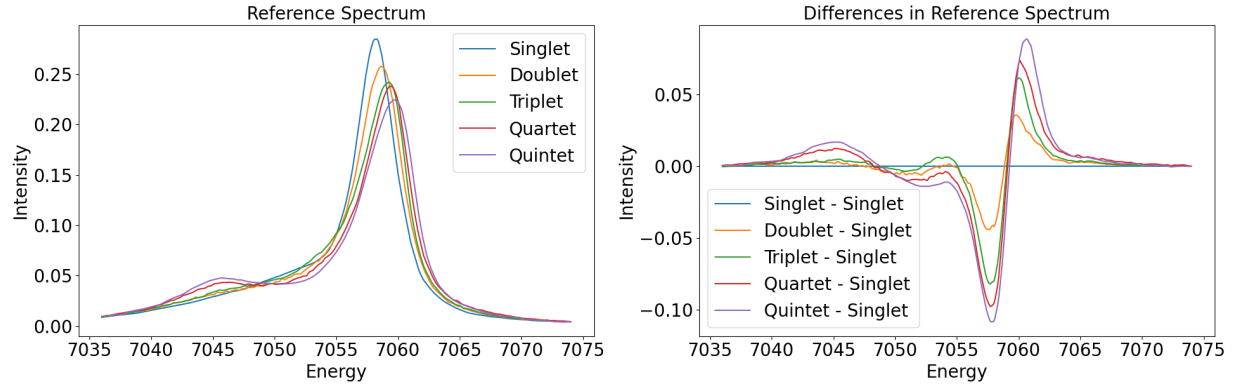


Figure 16: The reference spectra plotted with provided data for different spin states. The reference difference spectra with transient signals plotted with provided data for different transitions.

5.1. XES spectra at 100ps

When plotting the XES spectrum and the difference spectrum for a time delay of 100 ps, a clear transient signal is recognisable with the expected peaks from the reference spectrum and the XES spectrum shows similarities with the singlet spectrum from the reference (see Figure 17).

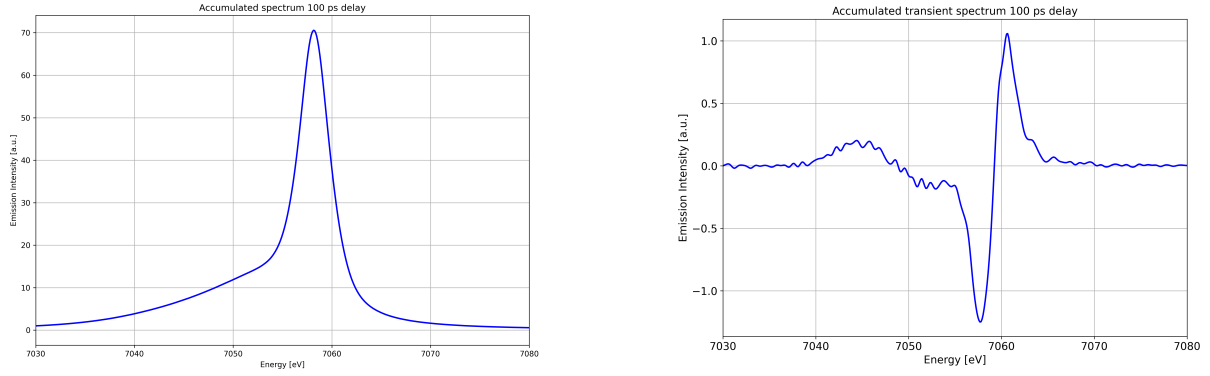


Figure 17: On the left the XES spectrum with the excitation laser on at a delay of 100 ps is plotted. On the right there's the difference spectrum at the same time delay.

5.2. Difference spectra for different pump-probe delays

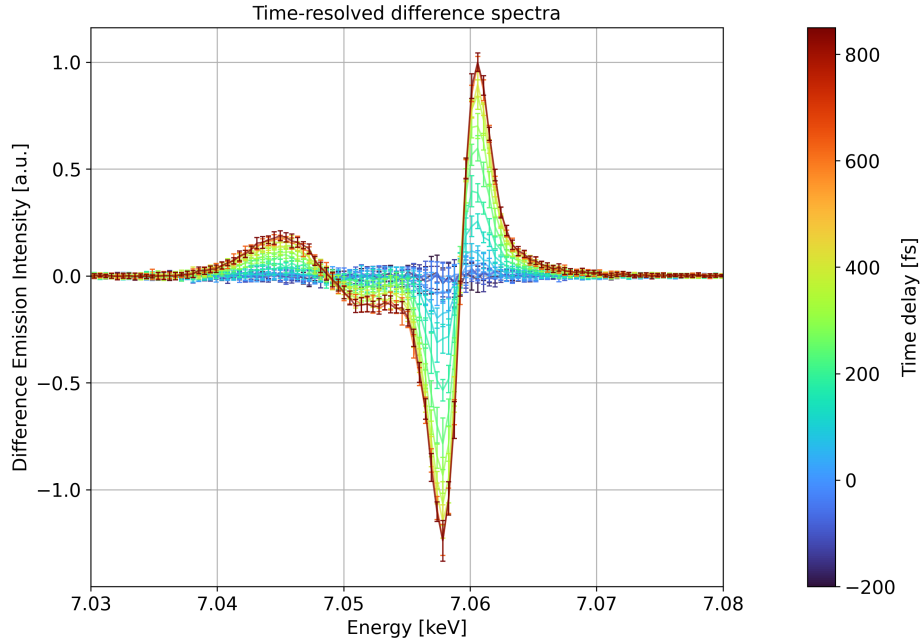


Figure 18: Transient spectra of the X-ray emission at different pump-probe time delays.

From the time resolved difference spectra of the measurements (see Figure 18) we can see multiple developments in the transient signal over the different time delays: at an energy around 7045.0 eV the difference emission intensity grows (the intensity for the excited state is more intense than that of the ground state), at about 7053.7 eV a shoulder in intensity difference initially emerges and then flattens out again for time delays $\Delta t > 350$ fs. The two most prominent features of the transient signal are the minimum at 7057.8 eV and the maximum at 7060.5 eV which both match up with the difference spectrum provided.

Another thing to note is how little the change in spectrum between 600 fs and 800 fs is. The reaction has clearly landed on a more stable state, even though we can tell that it is still an excited state when comparing to the reference spectrum as it most closely matches the quintet-singlet transition denoted there. The comparison with the reference spectrum also tells us that there likely is a short-lived intermediate state as the shoulder we see resembles the triplet or quartet spectra.

The kinetic traces at these energies (see Figure 19) can be used to determine the rise time of the

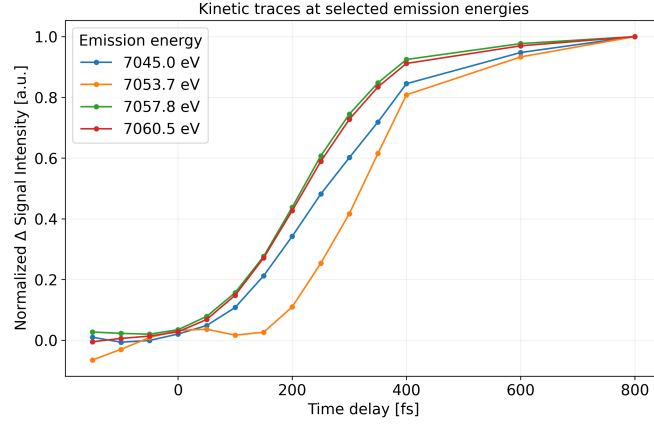


Figure 19: Kinetic traces at selected emission energies.

population of the excited states of $[\text{Fe}(\text{bipy})_3]^{2+}$ during the relaxation. To do this we use a function to parametise our reference spectrum:

$$S(E) = A \cdot \sum_{s=1}^5 f_s \cdot S_s(E) \quad (13)$$

With A being the amplitude, $s = \{1, 2, 3, 4, 5\}$ describing the multiplicity and f_s being the population coefficients of the corresponding reference spectra $S_s(E)$. This means the population of the ground state - the singlet state - is described through f_1 .

The population T of our difference spectrum can now be described as:

$$T(E) = S(E) - S_1(E) = A \cdot \left(\sum_{s=1}^5 f_s \cdot S_s(E) - S_1(E) \right) \quad (14)$$

The fitting function can be found in ??.

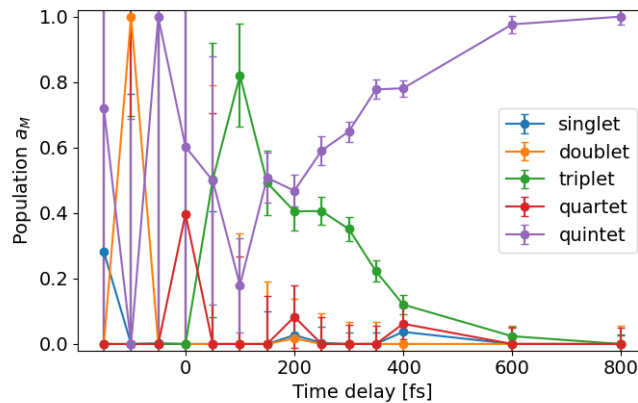


Figure 20: The population curves of the different spin states plotted against the time delay of the optical laser in femtoseconds.

Now that we have a fit for the time-dependant population of the respective states present during the different time delays (see Figure 20), we can see that our population curves have huge errorbars for the time delays $\Delta t \leq 0$ (better represented in Figure 25). This explains the occurrence of any spin multiplicity other than singlet for our negative delays, as they should not occur. Our data is noisy for these delays

and the fitting function struggles to get a read on the data.

To confirm this, we looked at the squared errors over the negative time delays (see Figure 22) and surmised that our photon count was much too low to justify the peaks seen in our population curve. This is also corroborated by looking at the residuals together with the measurements and seeing how noisy the measurements for a time delay of $\Delta t \leq 0$ are in comparison to the one at 50 fs (see Figure 21).

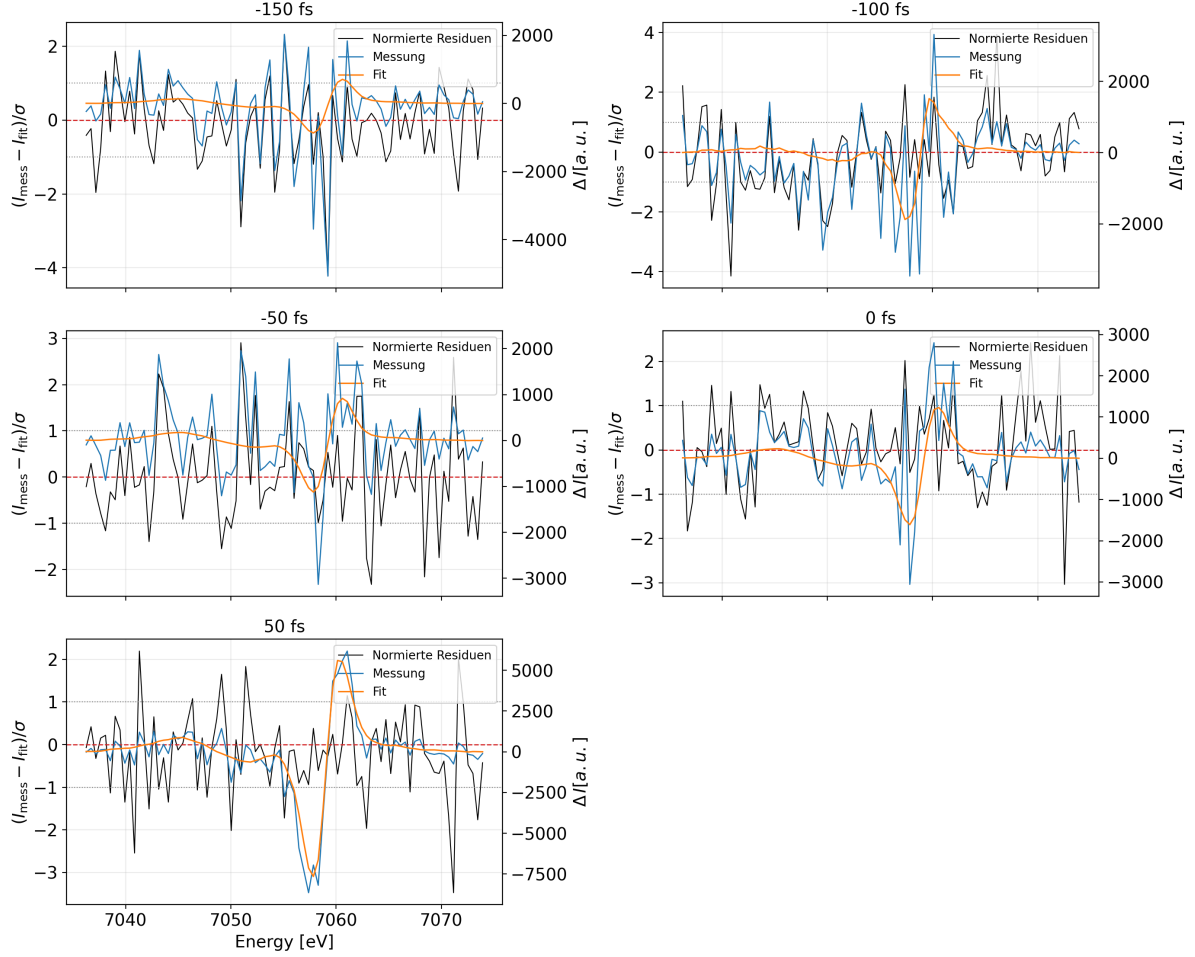


Figure 21: The normalised residuals together with the actual measurements and a fit over ... for time delays of $\Delta t = \{-150 \text{ fs}, -100 \text{ fs}, -50 \text{ fs}, 0 \text{ fs}, 50 \text{ fs}\}$

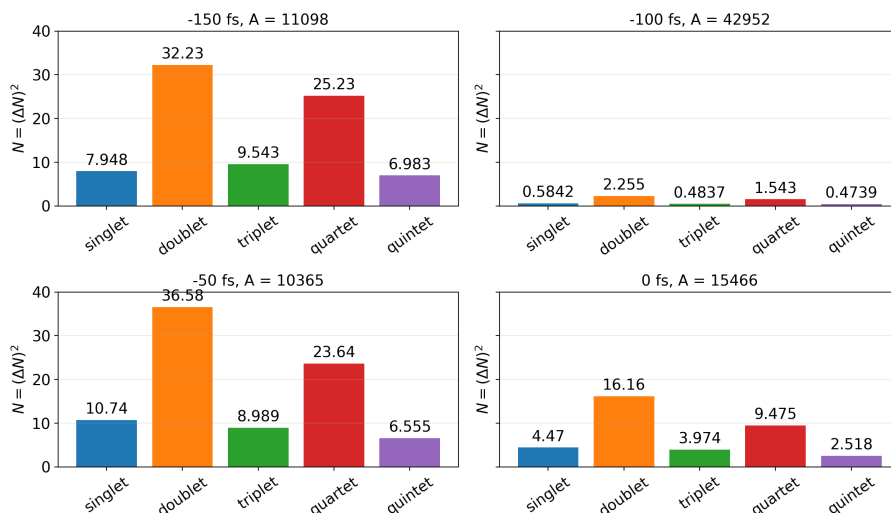


Figure 22: The photon counts of the different excited states for negative time delays.

We then used this information to fix our population curves (see Figure 23). This new function (see: ??) fits the state population for delays up to 50 fs simultaneously while softly constraining them to remain close to a common baseline, thereby stabilizing the fit for the weak pre-time-zero signals.

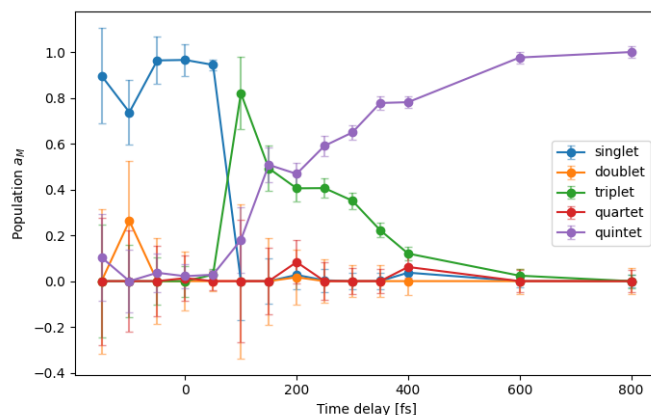


Figure 23: The adjusted population curves of the different spin states plotted against the time delay of the optical laser in femtoseconds.

With this version of our fit we can see that nothing really changed for our positive delays. We see a population change from majority singlet to almost no singlets around our 50 fs and 100 fs delay with only small peaks at later delays with errorbars too big to really be definite peaks. Our triplet population grows rapidly for our 100 fs delay. After that peak our triplet population steadily decreases and is overtaken by the quintet population which grows steadily from a delay of 50 fs. At our latest time delay of 800 fs the population is made up of almost only quintets with all other states being near 0. The doublet and quartet populations are near 0 for almost all time delays which is to be expected with the predicted intermediate states of the reaction.

6. Discussion of the physical meaning

In both our versions of our population curves we can see the triplet population spike. This is indicative of an ultrafast intermediate state - the 3T_1 state - in the relaxation chain. Approximately 80% of our excited molecules had a relaxation chain of $^{1,3}\text{MLCT} \rightarrow ^3T_1 \rightarrow ^5T_2$ while about 20% directly transitioned from the $^{1,3}\text{MLCT}$ state into the high spin 5T_2 state.

The relaxation timescale matches roughly to our literature value of $\tau < 100$ fs, even though with a peak at 100 fs it's a bit later than expected. This could have happened because of an error in determining our origin (no time delay).

While talking about our relaxation timeline we can also see that in our chosen time delays the relaxation doesn't fully end, as we end at a maximal quintet population at our time delay of 800 fs and with a difference spectrum that still shows small differences when compared to the difference spectrum at 100 ps but not majorly so. This matches our literature value for the relaxation time of the 5T_2 state being 1 ns which is much larger than our latest time delay but not that much more bigger than our 100 ps measurement.

In our population curves we also see doublet and quartet states, which shouldn't necessarily be part of our relaxation chain. The doublets can occur because the whole molecule can look like a doublet. But they, like our quartets only really occur because of errors in our fitting function which struggles with strongly correlated data for low photon counts.

These errors in our fitting function are also the reason that our first population curve looks the way it does. The algorithm struggles with the noise present at low photon counts.

6.1. Error analysis

Our errorbars for our difference spectrum measurement are a result of the statistical nature of the data which the one kind of error that is programmed into VLab. They have a size that is about two orders of magnitude smaller than the highest peak in arbitrary units. To get smaller errorbars by half we would have to accumulate the difference spectra for four times the length of time which wasn't feasible on the day we took our measurements especially since our program had crashed two times already and we preferred getting all the data we wanted with suboptimal errors over risking getting only part of the time delays done.

7. Conclusion and outlook

During the experiment the X-ray beam was successfully characterised and focussed to a usable point with a FWHM of $13.4\ \mu\text{m}$. The following settings for the pump-laser were determined:

- laser power: W
- rep rate: 550 000 Hz
- pulse energy: J
- wavelength: 515 nm
- full width vertical: $150\ \mu\text{m}$
- full width horizontal: $150\ \mu\text{m}$

Then a successful series of pump-probe-measurements was carried out in which we can see the occurrence of an ultrafast intermediate state can be seen. The identification was carried out using a provided reference spectrum with which it was possible to fit time delay-dependant population curves resolved after the different spin states present during the relaxation process.

The experiment successfully provided insight into the work on a real beamline.

References

- [1] P. Willmott, *An Introduction to Synchrotron Radiation*. Wiley, 2019.
- [2] A. Owens, “Synchrotron light sources and radiation detector metrology,” *Nuclear Instruments and Methods in Physics Research Section A: Accelerators, Spectrometers, Detectors and Associated Equipment*, vol. 695, pp. 1–12, 2012.
- [3] R. L. K. Kim, Z. Huang, *Synchrotron Radiation and Free-Electron Lasers - Principles of Coherent X-Ray Generation*. Cambridge University Press, 2017.
- [4] C. Bressler, “Ultrafast x-ray experiments virtual experiments at the x-ray laser european xfel.”
- [5] S. Hayama, G. Duller, J. Sutter, M. Amboage, R. Boada, A. Freeman, L. Keenan, B. Nutter, L. Cahill, P. Leicester, B. Kemp, N. Rubies, and S. Diaz-Moreno, “The scanning four-bounce monochromator for beamline i20 at the diamond light source,” *Journal of Synchrotron Radiation*, vol. 25, pp. 1556–1564, 08 2018.

List of Figures

1.	A schematic of a pump-probe-experiment [4].	2
2.	Schematic view of $[\text{Fe}(\text{bipy})_3]^{2+}$	3
3.	Energy level diagram of the d orbitals of a d^6 transition metal ion in an octahedral crystal field. An assumed spherical ligand field raises the energy of the d orbitals uniformly by approximately 20 eV to 40 eV. The degeneracy of the d orbitals is removed by the actual octahedral field of the point-charged ligands. [4]	3
4.	The potential energy curves of the excited state dynamics of $[\text{Fe}(\text{bipy})_3]^{2+}$ in H_2O after the excitation to the MLCT band. [4].	4
5.	Optical layout of a four-crystal monochromator [5].	5
6.	The optical bench in the FXE hut with the following components implemented: BIU 1,2,3 (beam imaging units), SpA (spectrum analyser), TAD (time arrival detector), PS 1,2 (power slits), SAA (solid attenuator assembly), IPM1 (intensity position monitor), CRLs (compound refractive lenses) [4].	5
7.	X-Ray absorption spectrum of $[\text{Fe}(\text{terpy})_2]^{2+}$ (terpy: terpyridin) at the Fe K edge displaying the XANES and EXAFS regions including the background absorption contributions μ_0 . The inset zooms into the weak pre edge features [4].	7
8.	To the left is the attenuated pink beam, to the right the monochromatic beam with the same attenuators in.	8
9.	To the left is a 3D intensity plot of the pink beam, to the right the 3D intensity plot of the monochromatic beam with the same attenuators in.	8
10.	The horizontal and vertical profile of the focussed X-ray beam plotted together with a gaussian fit and the corresponding FWHM of $13.5 \mu\text{m}$	9
11.	UV/VIS absorption spectrum of an aqueous $[\text{Fe}(\text{bipy})_3]\text{Cl}_2$ solution. The different CT transitions are indicated. The measured molar absorptivity ϵ values lie in the expected range of $10^{-4} \text{ l mol}^{-1} \text{ cm}^{-1}$ to $10^{-5} \text{ l mol}^{-1} \text{ cm}^{-1}$ for dipole-allowed transitions[4]	10
12.	The Von Hamos spectrometer in operating position. with the read-out of the Jungfrau detector as well as the settings for the position to the left.	12
13.	The profile of the X-ray beam measured in the sample plane.	12
14.	The profile of the laser beam measured in the sample plane.	12
15.	The profile of the overlapping laser and X-ray beams in the sample plane.	13
16.	The reference spectra plotted with provided data for different spin states. The reference difference spectra with transient signals plotted with provided data for different transitions.	13
17.	On the left the XES spectrum with the excitation laser on at a delay of 100 ps is plotted. On the right there's the difference spectrum at the same time delay.	14
18.	Transient spectra of the X-ray emission at different pump-probe time delays.	14
19.	Kinetic traces at selected emission energies.	15
20.	The population curves of the different spin states plotted against the time delay of the optical laser in femtoseconds.	15
21.	The normalised residuals together with the actual measurements and a fit over ... for time delays of $\Delta t = \{-150 \text{ fs}, -100 \text{ fs}, -50 \text{ fs}, 0 \text{ fs}, 50 \text{ fs}\}$	16
22.	The photon counts of the different excited states for negative time delays.	17
23.	The adjusted population curves of the different spin states plotted against the time delay of the optical laser in femtoseconds.	17
24.		C
25.	The population curves of the different spin states plotted against the time delay of the optical laser in femtoseconds with another y-axes scale to see the errorbars better.	C

Appendix A Additional Plots

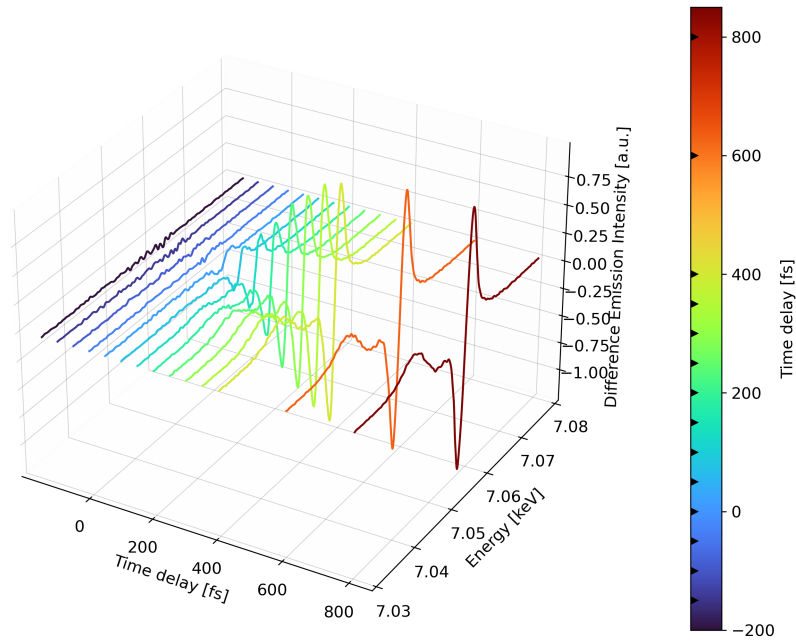


Figure 24

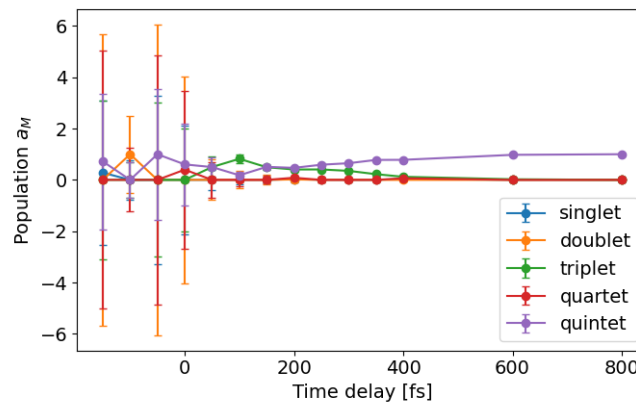


Figure 25: The population curves of the different spin states plotted against the time delay of the optical laser in femtoseconds with another y-axes scale to see the errorbars better.

Appendix B Code

Listing 1: Time resolved difference spectra (2D)

```

1 import numpy as np
2 import pandas as pd
3 import matplotlib.pyplot as plt
4 from pathlib import Path
5 from scipy.interpolate import interp1d
6 from matplotlib import cm
7 from matplotlib.colors import Normalize
8
9 plt.rcParams.update({
10     "font.size": 14,

```

```

11     "axes.titlesize": 14,
12     "axes.labelsize": 14,
13     "xtick.labelsize": 14,
14     "ytick.labelsize": 14,
15     "legend.fontsize": 14,
16     "legend.title_fontsize": 14,
17 })
18
19 folder = Path("Daten/Experiment with Data Acquisition/messreihe")
20 folders = sorted(folder.glob("*"))
21
22 all_spectra = [] # mean per delay on E_ref
23 all_sd = [] # sd per delay on E_ref
24 all_Eref = [] # raw energy grid per delay (chosen from first file)
25 delays = []
26
27 for subfolder in folders:
28
29     y_interp_list = []
30     E_ref = None
31
32     # exactly 10 files
33     files = sorted(subfolder.glob("*"))
34     if len(files) == 0:
35         continue
36
37     for file in files:
38         df = pd.read_table(
39             file,
40             sep=r'\s+',
41             comment='#',
42             header=None,
43             names=['Number', 'Energy', 'Intensity']
44         )
45
46         E = df["Energy"].to_numpy()
47         y = df["Intensity"].to_numpy()
48
49         # Ensure increasing E for interp1d
50         idx = np.argsort(E)
51         E = E[idx]
52         y = y[idx]
53
54         if E_ref is None:
55             E_ref = E # choose raw measured energies from first file
56
57         interp_func = interp1d(
58             E, y,
59             kind="cubic",
60             bounds_error=False,
61             fill_value=np.nan
62         )
63         y_interp_list.append(interp_func(E_ref))
64
65     if E_ref is None or len(y_interp_list) == 0:
66         continue
67
68     spectra_interp = np.array(y_interp_list)
69     y_average = np.nanmean(spectra_interp, axis=0)
70     sd = np.nanstd(spectra_interp, axis=0, ddof=1)
71
72     # Convert folder name to numerical delay
73     name = subfolder.name.lower().replace("fs", "").strip()
74     delay = (
75         -float(name.replace("min", "").strip())
76         if name.startswith("min")
77         else float(name)

```

```

78     )
79
80     delays.append(delay)
81     all_spectra.append(y_average)
82     all_sd.append(sd)
83     all_Eref.append(E_ref)
84
85     delays = np.array(delays)
86     sort_idx = np.argsort(delays)
87
88     all_spectra = np.array(all_spectra)
89     all_sd = np.array(all_sd)
90     all_Eref = np.array(all_Eref)
91
92     delays = delays[sort_idx]
93     all_spectra = all_spectra[sort_idx]
94     all_sd = all_sd[sort_idx]
95     all_Eref = all_Eref[sort_idx]
96
97     all_spectra = all_spectra * 1e-5 # for better readability
98     all_sd = all_sd * 1e-5
99
100 # 3d color plot
101 fig, ax = plt.subplots(figsize=(10, 7))
102
103 norm = Normalize(vmin=delays.min(), vmax=delays.max())
104 cmap = cm.turbo
105
106 # option to sparsify error bars so it doesn't become unreadable
107 num_points = len(E_ref)
108
109 for delay, E_ref, spectrum, sd in zip(delays, all_Eref, all_spectra, all_sd):
110     color = cmap(norm(delay))
111
112     inds = np.linspace(0, len(E_ref) - 1, num_points, dtype=int)
113
114     # error bars on raw measured points
115     ax.errorbar(
116         E_ref[inds] * 0.001,      # keV
117         spectrum[inds],
118         yerr=sd[inds],
119         fmt="none",
120         ecolor=color,
121         elinewidth=1.0,
122         capsize=2,
123         alpha=0.9
124     )
125
126     # optional: plot the mean curve
127     ax.plot(E_ref * 0.001, spectrum, color=color, linewidth=1.2, alpha=0.7)
128
129 ax.set_xlabel("Energy [keV]")
130 ax.set_ylabel("Difference Emission Intensity [a.u.]")
131 ax.set_xlim(7.030, 7.080)
132 ax.set_title("Time-resolved difference spectra")
133
134 sm = cm.ScalarMappable(norm=norm, cmap=cmap)
135 sm.set_array([])
136 cbar = fig.colorbar(sm, ax=ax, pad=0.1)
137 cbar.mappable.set_clim(min(delays) - 50, max(delays) + 50)
138 cbar.set_label("Time delay [fs]")
139
140 plt.tight_layout()
141 plt.grid()
142
143 plt.savefig(
144     "Daten/Analysis and Interpretation/Time_resolved_spectrum_errbars.png",

```

```

145     dpi=300
146 )
147
148 plt.show()

```

Listing 2: kinetic traces plots

```

1 import numpy as np
2 import pandas as pd
3 import matplotlib.pyplot as plt
4 from pathlib import Path
5 from scipy.interpolate import interp1d
6 from scipy.ndimage import gaussian_filter1d
7 from scipy.optimize import curve_fit
8 from scipy.special import erf
9
10 plt.rcParams.update({
11     "font.size": 14,
12     "axes.titlesize": 14,
13     "axes.labelsize": 14,
14     "xtick.labelsize": 14,
15     "ytick.labelsize": 14,
16     "legend.fontsize": 14,
17     "legend.title_fontsize": 14,
18 })
19
20 folder = Path("Daten/Experiment with Data Acquisition/messreihe")
21
22 folders = sorted(folder.glob("*"))
23
24 all_spectra = []
25 delays = []
26
27 # Common energy grid
28 energy_grid= np.linspace(7000, 7110, 1000)
29
30 for subfolder in folders:
31
32     spectra = []
33
34     for file in subfolder.glob("*"):
35
36         df = pd.read_table(
37             file,
38             sep=r'\s+',
39             comment='#',
40             header=None,
41             names=['Number', 'Energy', 'Intensity']
42         )
43
44         E = df['Energy'].to_numpy()
45         y = df['Intensity'].to_numpy()
46
47         # Interpolate onto common energy grid
48         interp_func = interp1d(
49             E,
50             y,
51             kind='cubic',
52             bounds_error=False,
53             fill_value=np.nan
54         )
55
56         y_interp = interp_func(energy_grid)
57
58         spectra.append(y_interp)
59
60 spectra = np.array(spectra)

```

```

61
62     # Mean over the 20 measurements
63     y_average = np.nanmean(spectra, axis=0)
64
65     # Standard deviation over the 20 measurements
66     sd = np.nanstd(spectra, axis=0, ddof=1)
67
68     all_spectra.append(y_average)
69
70     # Convert folder name to numerical delay
71     name = subfolder.name.lower().replace("fs", "").strip()
72     delay = (
73         -float(name.replace("min", "").strip())
74         if name.startswith("min")
75         else float(name)
76     )
77     delays.append(delay)
78
79 all_spectra = np.array(all_spectra)
80 delays = np.array(delays)
81
82 # Sort by delay
83 sort_idx = np.argsort(delays)
84
85 delays = delays[sort_idx]
86 print(delays)
87 all_spectra = all_spectra[sort_idx]*1e-5 #Intensity is in a.u., so it is scaled down for
      better readability in the plot (*10^5)
88
89 # KINETIC TRACES
90 intensity = all_spectra
91
92 # Smooth only in TIME direction
93 intensity_smooth = gaussian_filter1d(intensity, sigma=1.0, axis=0)
94
95 # Plot the four kinetic traces
96 fig, ax = plt.subplots(figsize=(10, 6))
97
98 selected_energies = [
99     7045.0,
100    7053.7,
101    7057.8,
102    7060.5
103 ]
104
105 # Find the closest available energy-grid point
106 selected_indices_manuel = [
107     np.argmin(np.abs(energy_grid - E))
108     for E in selected_energies
109 ]
110 print(selected_indices_manuel)
111
112 for idx in selected_indices_manuel:
113     trace = intensity_smooth[:, idx]
114
115     # Invert selected traces
116     if np.isclose(energy_grid[idx], 7053.7, atol=0.1) or \
117        np.isclose(energy_grid[idx], 7057.8, atol=0.1):
118         trace = -trace
119
120     # Normalize ?
121     trace_norm = trace / np.max(np.abs(trace)) # For comparing timing/shape:
122     # trace_norm = trace / np.max(np.abs(intensity_smooth)) # For comparing actual
123     # magnitude of the diff-Intensity:
124
125     ax.plot(
126         delays,

```

```

126         trace_norm,
127         marker="o",
128         markersize=4,
129         linewidth=1.5,
130         label=f"{energy_grid[idx]:.1f} eV"
131     )
132
133 ax.set_xlabel("Time delay [fs]")
134 ax.set_ylabel("Normalized  $\Delta$  Signal Intensity [a.u.]")
135 ax.set_title("Kinetic traces at selected emission energies")
136 ax.legend(title="Emission energy")
137 ax.grid(True, alpha=0.2)
138 plt.savefig(
139     "Daten/Analysis and Interpretation/Kinetic traces.png",
140     dpi=300
141 )
142
143 # Choose the energy that represents the excited-state population
144 target_energy = 7045 #? oder eher 7053,6?
145 # TODO: find out welcher fit
146
147 # Rise time by 10-90% fit (bis Zeile 230)
148 idx = np.argmin(
149     np.abs(energy_grid - target_energy)
150 )
151 trace = intensity_smooth[:, idx]
152 # Invert if necessary
153 trace = -trace
154 # Normalize
155 trace = trace / np.max(np.abs(trace))
156
157 # Baseline: average of measurements before time zero
158 pre_mask = delays < 0
159 baseline = np.mean(trace[pre_mask])
160 # Final population: average of the last few measurements
161 final_value = np.mean(trace[-2:])
162
163 # 10 % and 90 % of the population rise
164 level_10 = baseline + 0.10 * (final_value - baseline)
165 level_90 = baseline + 0.90 * (final_value - baseline)
166
167 '''print("Rise time of excited-state population")
168 print(f"Emission energy: {energy_grid[idx]:.2f} eV")
169 print(f"Baseline: {baseline:.3f}")
170 print(f"Final value: {final_value:.3f}")
171 print(f"10 % level: {level_10:.3f}")
172 print(f"90 % level: {level_90:.3f}")
173 '''
174 # Find crossing of 10 % and 90 %
175
176 # Only consider times after time zero
177 post_mask = delays >= 0
178
179 t = delays[post_mask]
180 I = trace[post_mask]
181
182 def find_crossing_time(t, I, level):
183     # Find first point above the requested level
184     crossing_indices = np.where(I >= level)[0]
185
186     if len(crossing_indices) == 0:
187         return np.nan
188
189     i = crossing_indices[0]
190
191     # If the first point is already above the level
192     if i == 0:

```

```

193         return t[0]
194
195     # Linear interpolation between the two surrounding points
196     t1 = t[i - 1]
197     t2 = t[i]
198
199     I1 = I[i - 1]
200     I2 = I[i]
201
202     t_cross = t1 + ((level - I1) / (I2 - I1) * (t2 - t1))
203
204     return t_cross
205
206
207 t10 = find_crossing_time(t, I, level_10)
208 t90 = find_crossing_time(t, I, level_90)
209 rise_time = t90 - t10
210
211 # Plot
212 plt.figure(figsize=(10, 6))
213
214 plt.plot(
215     delays,
216     trace,
217     "o-",
218     #label=f"{energy_grid[idx]:.1f} eV"
219     label=f"Fit, rise time = {rise_time:.1f} fs"
220 )
221
222 # 10 % and 90 % levels
223 plt.axhline(level_10, linestyle="--", alpha=0.6, label=f"10 % ({t10:.1f} fs)")
224 plt.axhline(level_90, linestyle="--", alpha=0.6, label=f"90 % ({t90:.1f} fs)")
225
226 # Vertical lines at crossing times
227 plt.axvline(t10, linestyle=":", alpha=0.6)
228 plt.axvline(t90, linestyle=":", alpha=0.6)
229
230 plt.xlabel("Time delay [fs]")
231 plt.ylabel("Normalized Emission Intensity")
232 plt.legend()
233 plt.title(
234     f"Rise time (10-90%-Fit) at {energy_grid[idx]:.1f} eV"
235 )
236 plt.savefig(
237     "Daten/Analysis and Interpretation/population rise time at 7045eV_1090fit.png",
238     dpi=300)
239
240 # Rise function with Sigmoid-Fit
241 def rise_function(t, A, t0, sigma, B):
242     return (B + A * 0.5 * (1 + erf((t - t0) / (np.sqrt(2) * sigma))))
243
244 # Fit
245 p0 = [
246     1.0,      # A
247     200.0,    # t0
248     150.0,    # sigma
249     0.0       # B
250 ]
251 popt, pcov = curve_fit(rise_function, delays, trace, p0=p0, maxfev=20000)
252 A, t0, sigma, B = popt
253
254 # Calculate 10-90 % rise time
255 # For an error-function rise:
256 # 10 % -> -1.28155 sigma
257 # 90 % -> +1.28155 sigma
258 # Therefore:
259

```

```

260 rise_time = 2 * 1.28155 * sigma
261
262 print(f"Energy: {energy_grid[idx]:.1f} eV")
263 print(f"t0           = {t0:.2f} fs")
264 print(f"sigma        = {sigma:.2f} fs")
265 print(f"Rise time    = {rise_time:.2f} fs")
266
267 # Plot fit
268 t_dense = np.linspace(delays.min(),delays.max(),1000)
269 fit = rise_function(t_dense,*popt)
270
271 plt.figure(figsize=(10, 6))
272 plt.plot(delays,trace,"o",label="Measurement")
273 plt.plot(t_dense,fit,linewidth=2,label=f"Fit, rise time = {rise_time:.1f} fs")
274
275 plt.xlabel("Time delay [fs]")
276 plt.ylabel("Normalized Emission Intensity")
277 plt.title(
278     f"Rise time (Sigmoid-Fit) at {energy_grid[idx]:.1f} eV"
279 )
280
281 plt.legend()
282 plt.grid(True, alpha=0.2)
283 plt.savefig(
284     "Daten/Analysis and Interpretation/population rise time at 7045eV_sigmoidfit.png",
285     dpi=300)
286
287 plt.tight_layout()
288 plt.show()

```

Listing 3: Fit for population curves

```

1 import numpy as np
2 import matplotlib.pyplot as plt
3 from pathlib import Path
4 from scipy.interpolate import interp1d
5 from scipy.optimize import minimize
6 plt.rcParams.update({
7     "font.size": 14,
8     "axes.titlesize": 14,
9     "axes.labelsize": 14,
10    "xtick.labelsize": 14,
11    "ytick.labelsize": 14,
12    "legend.fontsize": 14,
13    "legend.title_fontsize": 14,
14 })
15
16 def load_spectrum(file):
17     data = np.loadtxt(
18         file,
19         #sep=r'\s+',
20         comments="#"
21     )
22     number = data[:, 0]
23     energy = data[:, 1]
24     intensity = data[:, 2]
25
26     return energy, intensity
27
28 def load_reference_spectra(ref_file):
29
30     data = np.loadtxt(
31         ref_file,
32         delimiter=",",
33         comments="#",
34         skiprows=1
35     )

```

```

36
37     energy_ref = data[:, 0]
38     reference_spectra = data[:, 1:6]
39
40     return energy_ref, reference_spectra
41
42 def build_deltaI_matrix_from_folders(
43     data_folder: Path,
44     delays,
45     delay_names,
46     load_spectrum,
47 ):
48     all_means = []
49     all_sigmas = []
50     t_fs_list = []
51
52     energy_common = None
53
54     for delay, delay_name in zip(delays, delay_names):
55         folder = data_folder / delay_name
56         files = sorted(folder.glob("*"))
57
58         spectra_interp = []
59         energies = []
60
61         #print(f"Delay: {delay} fs")
62
63         for file in files:
64             energy, intensity = load_spectrum(file)
65             energies.append(energy)
66
67         #common energy grid
68         energy_common = energies[0]
69         spectra = []
70
71         #interpolate to common grid
72         for file in files:
73             energy, intensity = load_spectrum(file)
74             f = interp1d(energy, intensity, bounds_error=False, fill_value=np.nan)
75             spectra.append(f(energy_common))
76
77         spectra = np.array(spectra) # (n_shots, NE)
78
79         mean_intensity = np.nanmean(spectra, axis=0) # (NE,)
80         std_intensity = np.nanstd(spectra, axis=0, ddof=1)
81
82         n_shots = np.sum(~np.isnan(spectra[:, 0]))
83
84         #assume same n_shots across bins:
85         sem_intensity = std_intensity / np.sqrt(len(spectra))
86
87         #avoid sigma=0 bc no dividing by zero
88         sigma = sem_intensity.copy()
89         positive_sigma = sigma[sigma > 0]
90         minimum_sigma = np.min(positive_sigma) if len(positive_sigma) > 0 else 1.0
91         sigma[sigma <= 0] = minimum_sigma
92
93         all_means.append(mean_intensity)
94         all_sigmas.append(sigma)
95         t_fs_list.append(delay)
96
97     dI_mean = np.array(all_means)
98     sigma = np.array(all_sigmas)
99     t_fs = np.array(t_fs_list)
100
101     return t_fs, energy_common, dI_mean, sigma
102

```

```

103 def fit_difference_spectrum(energy, difference, sigma, reference_spectra):
104     #Fit  $D(E) = A * (\sum(a_M * S_M(E)) - S_1(E))$ 
105     ground_state = reference_spectra[:, 0]
106
107     def model(parameters):
108         amplitude = parameters[0]
109         excited_populations = parameters[1:]
110         singlet_population = 1.0 - np.sum(excited_populations)
111         populations = np.concatenate([singlet_population, excited_populations])
112         return amplitude * (reference_spectra @ populations - ground_state), populations
113
114     def objective(parameters):
115         fitted, _ = model(parameters)
116         return np.sum(((difference - fitted) / sigma) ** 2)
117
118     # für alle states
119     #x0 = np.array([1.0, 0.1, 0.1, 0.1, 0.1])
120     x0 = np.array([0.1, 1.0, 0.1, 0.1, 0.0])
121     result = minimize(
122         objective,
123         x0,
124         method="SLSQP",
125         bounds=[(0.0, None), (0.0, 1.0), (0.0, 1.0), (0.0, 1.0), (0.0, 1.0)],
126         constraints={"type": "ineq", "fun": lambda parameters: 1.0 - np.sum(parameters
127         [1:])},
128         options={"ftol": 1e-12, "maxiter": 2000},
129     )
130     amplitude = result.x[0]
131     fitted, populations = model(result.x)
132     return amplitude, populations, fitted, result
133
134 def estimate_population_uncertainties(result, difference, sigma, reference_spectra):
135     parameters = result.x
136     ground_state = reference_spectra[:, 0]
137
138     def weighted_residuals(parameters):
139         amplitude = parameters[0]
140         excited_populations = parameters[1:]
141         singlet_population = 1.0 - np.sum(excited_populations)
142         populations = np.concatenate([singlet_population, excited_populations])
143         fitted = amplitude * (reference_spectra @ populations - ground_state)
144         return (difference - fitted) / sigma
145
146     # Numerical Jacobian of the weighted residuals at the optimum
147     jacobian = np.empty((difference.size, parameters.size))
148     for parameter_index, parameter in enumerate(parameters):
149         step = np.sqrt(np.finfo(float).eps) * max(1.0, abs(parameter))
150         shifted_plus = parameters.copy()
151         shifted_minus = parameters.copy()
152         shifted_plus[parameter_index] += step
153         shifted_minus[parameter_index] -= step
154         jacobian[:, parameter_index] = (
155             weighted_residuals(shifted_plus) - weighted_residuals(shifted_minus)
156         ) / (2 * step)
157
158     covariance = np.linalg.pinv(jacobian.T @ jacobian)
159     population_transform = np.zeros((5, 5))
160     population_transform[0, 1:] = -1.0
161     population_transform[1:, 1:] = np.eye(4)
162     population_covariance = population_transform @ covariance @ population_transform.T
163     return np.sqrt(np.maximum(np.diag(population_covariance), 0.0))
164
165 def poisson_counts_for_negative_delays(t_fs, population_uncertainties, state_names):
166     """Calculate  $N = (\Delta N)^2$  from population error bars at  $t < 0$ """
167     negative_delays = t_fs <= 0
168     poisson_counts = population_uncertainties[negative_delays] ** 2

```

```

169
170     for delay, counts in zip(t_fs[negative_delays], poisson_counts):
171         print(f"{delay:>5.0f} fs: gemessene Anzahl/Intensität")
172         for state_name, count in zip(state_names, counts):
173             print(f"    {state_name:>8}: N = {count:.6g}")
174
175     return t_fs[negative_delays], poisson_counts
176
177 #---lets try it out---yipieyipieyipie
178 ref_file = Path("Daten/Analysis and Interpretation/Ref Spectrum/Reference_Data_Fig_4_4.
    csv")
179 data_folder = Path("Daten/Experiment with Data Acquisition/messreihe")
180
181 delays = [-150, -100, -50, 0, 50, 100, 150, 200, 250, 300, 350, 400, 600, 800]
182 delay_names = ["min150 fs", "min100 fs", "min50 fs", "0 fs", "50 fs", "100 fs", "150 fs",
    "200 fs", "250 fs", "300 fs", "350 fs", "400 fs", "600 fs", "800 fs"]
183
184 # deltaI matrix aus measurements
185 t_fs, energy_common, dI_mean, sigma = build_deltaI_matrix_from_folders(data_folder=
    data_folder, delays=delays, delay_names=delay_names, load_spectrum=load_spectrum)
186 # load and interpolate all five reference spectra
187 energy_ref, reference_ref = load_reference_spectra(ref_file)
188 reference_spectra = np.column_stack([
189     interp1d(energy_ref, reference_ref[:, state], bounds_error=False, fill_value=np.nan)(
        energy_common)
190     for state in range(reference_ref.shape[1])
191 ])
192
193 valid = (
194     np.isfinite(dI_mean).all(axis=0)
195     & np.isfinite(sigma).all(axis=0)
196     & np.isfinite(reference_spectra).all(axis=1)
197     & (sigma > 0).all(axis=0)
198 )
199
200 energy_fit = energy_common[valid]
201 dI_fit = dI_mean[:, valid]
202 sigma_fit = sigma[:, valid]
203
204 reference_fit = reference_spectra[valid]
205
206 amplitudes = []
207 populations = []
208 population_uncertainties = []
209 fitted_spectra = []
210 for delay, difference, uncertainty in zip(t_fs, dI_fit, sigma_fit):
211     amplitude, population, fitted, result = fit_difference_spectrum(energy_fit,
        difference, uncertainty, reference_fit)
212     amplitudes.append(amplitude)
213     populations.append(population)
214     population_uncertainties.append(estimate_population_uncertainties(result, difference,
        uncertainty, reference_fit))
215     fitted_spectra.append(fitted)
216     print(f"{delay:>5.0f} fs: A = {amplitude:.5g}, populations = {population}")
217
218 amplitudes = np.array(amplitudes)
219 populations = np.array(populations)
220 population_uncertainties = np.array(population_uncertainties)
221 fitted_spectra = np.array(fitted_spectra)
222
223 normalized_residuals = (dI_fit - fitted_spectra) / sigma_fit
224
225 state_names = ["singlet", "doublet", "triplet", "quartet", "quintet"]
226 negative_delays, poisson_counts = poisson_counts_for_negative_delays(t_fs,
    population_uncertainties, state_names)
227
228 plt.figure(figsize=(7, 4.5))

```

```

229 for index, state_name in enumerate(state_names):
230     plt.errorbar(
231         t_fs,
232         populations[:, index],
233         yerr=population_uncertainties[:, index],
234         fmt="o-",
235         capsize=3,
236         label=state_name,
237     )
238 plt.xlabel("Time delay [fs]")
239 plt.ylabel("Population $a_M$")
240 plt.ylim(-0.02, 1.02)
241 plt.legend()
242 plt.tight_layout()
243 plt.savefig("Auswertung EXP21/fit/actual fit/fit_all.png")
244 plt.show()
245
246 plt.figure(figsize=(7, 4.5))
247 for index, state_name in enumerate(state_names):
248     plt.plot(
249         negative_delays,
250         poisson_counts[:, index],
251         "o-",
252         label=state_name,
253     )
254 plt.xlabel("Time delay [fs]")
255 plt.ylabel(r"Measured intensity $N = (\Delta N)^2$")
256 plt.legend()
257 plt.tight_layout()
258 plt.savefig("Auswertung EXP21/fit/actual fit/poisson_counts_negative_delays.png")
259 plt.show()
260
261 # RESIDUEN UND SPEKTREN FÜR -150 BIS 50 fs
262 selected_indices = np.flatnonzero(t_fs <= 50)
263 selected_delays = t_fs[selected_indices]
264
265 number_of_delays = len(selected_delays)
266 number_of_columns = 2
267 number_of_rows = int(np.ceil(number_of_delays / number_of_columns))
268
269 fig, axes = plt.subplots(number_of_rows, number_of_columns, figsize=(14, 3.8 *
    number_of_rows), sharex=True, squeeze=False)
270 axes = axes.ravel()
271
272 for panel_index, fit_index in enumerate(selected_indices):
273     delay = t_fs[fit_index]
274     residual_axis = axes[panel_index]
275
276     # Normierte Residuen auf der linken y-Achse
277     residual_axis.plot(
278         energy_fit,
279         normalized_residuals[fit_index],
280         color="black",
281         linewidth=0.8,
282         label="Normierte Residuen")
283     residual_axis.axhline(
284         0,
285         color="tab:red",
286         linestyle="--",
287         linewidth=1)
288     residual_axis.axhline(
289         1,
290         color="gray",
291         linestyle=":",
292         linewidth=0.8)
293     residual_axis.axhline(
294         -1,

```

```

295         color="gray",
296         linestyle=":",
297         linewidth=0.8)
298
299     residual_axis.set_title(f"{delay:.0f} fs")
300     residual_axis.set_ylabel(r" $(I_{\mathrm{mess}} - I_{\mathrm{fit}}) / \sigma$ ")
301     residual_axis.grid(alpha=0.25)
302
303     # Spektren auf der rechten y-Achse
304     spectrum_axis = residual_axis.twinx()
305     measured_line, = spectrum_axis.plot(energy_fit, dI_fit[fit_index], color="tab:blue",
306                                         linewidth=1, label="Messung")
307     fitted_line, = spectrum_axis.plot(energy_fit, fitted_spectra[fit_index], color="tab:
308                                     orange", linewidth=1.2, label="Fit")
309     spectrum_axis.set_ylabel(r" $\Delta I$  [a.u.]")
310
311     # Gemeinsame Legende aus beiden Achsen
312     residual_lines, residual_labels = residual_axis.get_legend_handles_labels()
313     spectrum_lines, spectrum_labels = spectrum_axis.get_legend_handles_labels()
314     residual_axis.legend(residual_lines + spectrum_lines, residual_labels +
315                          spectrum_labels, loc="upper right", fontsize=10)
316
317     for axis in axes[number_of_delays:]:
318         axis.set_visible(False)
319
320     for axis in axes[-number_of_columns:]:
321         axis.set_xlabel("Energy [eV]")
322
323     fig.tight_layout()
324     fig.savefig("Auswertung EXP21/fit/actual fit/residuals_and_spectra_minus150_to_50fs.png",
325                dpi=200)
326     plt.show()
327
328     # COUNTS AT NEG DELAYS (ÜBER FEHLERBALKEN)
329     state_colors = plt.rcParams["axes.prop_cycle"].by_key()["color"][:len(state_names)]
330     number_of_negative_delays = len(negative_delays)
331     number_of_columns = 2
332     number_of_rows = int(np.ceil(number_of_negative_delays / number_of_columns))
333     fig, axes = plt.subplots(number_of_rows, number_of_columns, figsize=(12, 3.5 *
334                                number_of_rows), sharey=True, squeeze=False)
335     axes = axes.ravel()
336
337     negative_indices = np.flatnonzero(t_fs <= 0)
338     for panel_index, (delay, counts, fit_index) in enumerate(zip(negative_delays,
339                                                                    poisson_counts,
340                                                                    negative_indices)):
341         bars = axes[panel_index].bar(state_names, counts, color=state_colors)
342         axes[panel_index].set_title(f"{delay:.0f} fs, A = {amplitudes[fit_index]:.5g}")
343         axes[panel_index].set_ylabel(r" $N = (\Delta N)^2$ ")
344         axes[panel_index].tick_params(axis="x", rotation=35)
345         axes[panel_index].grid(axis="y", alpha=0.25)
346         axes[panel_index].bar_label(bars, fmt="%.4g", padding=3)
347         axes[panel_index].set_ylim(0.0, 40)
348
349     for axis in axes[number_of_negative_delays:]:
350         axis.set_visible(False)
351
352     fig.tight_layout()
353     #fig.savefig("Auswertung EXP21/fit/actual fit/counts_negative_delays_per_delay.png", dpi
354                =200)
355     plt.show()
356
357     # Poisson-Statistik
358     # Beim Zählen von Photonen mit einem Detektor misst man
359     # eine Intensität über die Anzahl der gezählten Ereignisse N
360     # Für diese Verteilung gilt eine fundamentale Eigenschaft:
361     # Varianz = Mittelwert ( = N)
362     # Da der Fehlerbalken N der Standardabweichung entspricht, gilt:

```

```

355 # Delta N = \sqrt{N}
356 # Quadriert man den Fehlerbalken, erhält man die gemessene Anzahl (Intensität):
357 # (Delta N)^2 = N

```

Listing 4: stabilized fit for negativ time delays

```

1 import numpy as np
2 import matplotlib.pyplot as plt
3 from pathlib import Path
4 from scipy.interpolate import interp1d
5 from scipy.optimize import minimize
6 from matplotlib.colors import to_rgba
7 '''plt.rcParams.update({
8     "font.size": 14,
9     "axes.titlesize": 14,
10    "axes.labelsize": 14,
11    "xtick.labelsize": 14,
12    "ytick.labelsize": 14,
13    "legend.fontsize": 14,
14    "legend.title_fontsize": 14,
15 })'''
16
17 def load_spectrum(file):
18     data = np.loadtxt(file, comments="#")
19     number = data[:, 0]
20     energy = data[:, 1]
21     intensity = data[:, 2]
22     return energy, intensity
23
24 def load_reference_spectra(ref_file):
25     data = np.loadtxt(ref_file, delimiter=",", comments="#", skiprows=1)
26     energy_ref = data[:, 0]
27     reference_spectra = data[:, 1:6]
28     return energy_ref, reference_spectra
29
30 def build_deltaI_matrix_from_folders(data_folder: Path, delays, delay_names,
31                                     load_spectrum):
32     all_means = []
33     all_sigmas = []
34     t_fs_list = []
35
36     energy_common = None
37
38     for delay, delay_name in zip(delays, delay_names):
39         folder = data_folder / delay_name
40         files = sorted(folder.glob("*"))
41         spectra_interp = []
42         energies = []
43
44         for file in files:
45             energy, intensity = load_spectrum(file)
46             energies.append(energy)
47
48         #common energy grid
49         energy_common = energies[0]
50         spectra = []
51
52         #interpolate to common grid
53         for file in files:
54             energy, intensity = load_spectrum(file)
55             f = interp1d(energy, intensity, bounds_error=False, fill_value=np.nan)
56             spectra.append(f(energy_common))
57
58         spectra = np.array(spectra)
59         mean_intensity = np.nanmean(spectra, axis=0)
60         std_intensity = np.nanstd(spectra, axis=0, ddof=1)

```

```

61     #assume same n_shots across bins:
62     sem_intensity = std_intensity / np.sqrt(len(spectra))
63
64     #avoid sigma=0 bc no dividing by zero
65     sigma = sem_intensity.copy()
66     positive_sigma = sigma[sigma > 0]
67     minimum_sigma = np.min(positive_sigma) if len(positive_sigma) > 0 else 1.0
68     sigma[sigma <= 0] = minimum_sigma
69
70     all_means.append(mean_intensity)
71     all_sigmas.append(sigma)
72     t_fs_list.append(delay)
73
74     dI_mean = np.array(all_means)
75     sigma = np.array(all_sigmas)
76     t_fs = np.array(t_fs_list)
77
78     return t_fs, energy_common, dI_mean, sigma
79
80 def fit_difference_spectrum(energy, difference, sigma, reference_spectra):
81     #Fit  $D(E) = A * (\sum(a_M * S_M(E)) - S_1(E))$ 
82     ground_state = reference_spectra[:, 0]
83
84     def model(parameters):
85         amplitude = parameters[0]
86         excited_populations = parameters[1:]
87         singlet_population = 1.0 - np.sum(excited_populations)
88         populations = np.concatenate([singlet_population, excited_populations])
89         return amplitude * (reference_spectra @ populations - ground_state), populations
90
91     def objective(parameters):
92         fitted, _ = model(parameters)
93         return np.sum(((difference - fitted) / sigma) ** 2)
94
95     # für alle states
96     #x0 = np.array([1.0, 0.1, 0.1, 0.1, 0.1])
97     x0 = np.array([0.1, 1.0, 0.1, 0.1, 0.0])
98     result = minimize(
99         objective,
100         x0,
101         method="SLSQP",
102         bounds=[(0.0, None), (0.0, 1.0), (0.0, 1.0), (0.0, 1.0), (0.0, 1.0)],
103         constraints={"type": "ineq", "fun": lambda parameters: 1.0 - np.sum(parameters
104 [1:])},
105         options={"ftol": 1e-12, "maxiter": 2000},
106     )
107
108     amplitude = result.x[0]
109     fitted, populations = model(result.x)
110     return amplitude, populations, fitted, result
111
112 def fit_negative_delays_soft(differences, sigmas, reference_spectra,
113     regularization_strength=10.0,):
114     """Fit separate populations while softly keeping them near one baseline."""
115     ground_state = reference_spectra[:, 0]
116     number_of_delays = differences.shape[0]
117     population_parameter_count = 4 * number_of_delays
118
119     def unpack(parameters):
120         excited_populations = parameters[:population_parameter_count].reshape(
121             number_of_delays, 4)
122         baseline = parameters[population_parameter_count:population_parameter_count + 4]
123         amplitudes = parameters[population_parameter_count + 4:]
124         populations = np.column_stack((1.0 - np.sum(excited_populations, axis=1),
125             excited_populations))
126         baseline_population = np.concatenate([1.0 - np.sum(baseline)], baseline))
127         return populations, baseline_population, amplitudes

```

```

124
125 def residuals(parameters):
126     populations, baseline_population, amplitudes = unpack(parameters)
127     spectral_shapes = reference_spectra @ populations.T - ground_state[:, None]
128     fitted = amplitudes[:, None] * spectral_shapes.T
129     data_residuals = ((differences - fitted) / sigmas).ravel()
130     population_residuals = np.sqrt(regularization_strength) * (populations[:, 1:] -
baseline_population[1:]).ravel()
131     return np.concatenate((data_residuals, population_residuals))
132
133 initial_amplitudes = np.maximum(np.max(np.abs(differences), axis=1), 1.0)
134 initial_populations = np.zeros((number_of_delays, 4))
135 x0 = np.concatenate((initial_populations.ravel(), np.zeros(4), initial_amplitudes))
136 bounds = (([0.0, 1.0]) * population_parameter_count + [[0.0, 1.0]] * 4 + [[0.0, None]
] * number_of_delays)
137
138 def population_constraints(parameters):
139     populations, baseline_population, _ = unpack(parameters)
140     return np.concatenate((populations[:, 0], [baseline_population[0]]))
141
142 result = minimize(
143     lambda parameters: np.sum(residuals(parameters) ** 2),
144     x0,
145     method="SLSQP",
146     bounds=bounds,
147     constraints={"type": "ineq", "fun": population_constraints},
148     options={"ftol": 1e-12, "maxiter": 2000},
149 )
150
151 populations, baseline_population, amplitudes = unpack(result.x)
152 spectral_shapes = reference_spectra @ populations.T - ground_state[:, None]
153 fitted = amplitudes[:, None] * spectral_shapes.T
154
155 jacobian = np.empty((residuals(result.x).size, result.x.size))
156 for parameter_index, parameter in enumerate(result.x):
157     step = np.sqrt(np.finfo(float).eps) * max(1.0, abs(parameter))
158     shifted_plus = result.x.copy()
159     shifted_minus = result.x.copy()
160     shifted_plus[parameter_index] += step
161     shifted_minus[parameter_index] -= step
162     jacobian[:, parameter_index] = (residuals(shifted_plus) - residuals(shifted_minus
)) / (2 * step)
163
164 covariance = np.linalg.pinv(jacobian.T @ jacobian)
165 population_uncertainties = np.empty((number_of_delays, 5))
166 for delay_index in range(number_of_delays):
167     covariance_block = covariance[4 * delay_index:4 * delay_index + 4,
168                                4 * delay_index:4 * delay_index + 4]
169     population_transform = np.vstack((-np.ones(4), np.eye(4)))
170     population_covariance = population_transform @ covariance_block @
population_transform.T
171     population_uncertainties[delay_index] = np.sqrt(
172         np.maximum(np.diag(population_covariance), 0.0)
173     )
174
175 return amplitudes, populations, fitted, population_uncertainties, result
176
177 def estimate_population_uncertainties(result, difference, sigma, reference_spectra):
178     parameters = result.x
179     ground_state = reference_spectra[:, 0]
180
181 def weighted_residuals(parameters):
182     amplitude = parameters[0]
183     excited_populations = parameters[1:]
184     singlet_population = 1.0 - np.sum(excited_populations)
185     populations = np.concatenate(([singlet_population], excited_populations))
186     fitted = amplitude * (reference_spectra @ populations - ground_state)

```

```

187         return (difference - fitted) / sigma
188
189     # Numerical Jacobian of the weighted residuals at the optimum
190     jacobian = np.empty((difference.size, parameters.size))
191     for parameter_index, parameter in enumerate(parameters):
192         step = np.sqrt(np.finfo(float).eps) * max(1.0, abs(parameter))
193         shifted_plus = parameters.copy()
194         shifted_minus = parameters.copy()
195         shifted_plus[parameter_index] += step
196         shifted_minus[parameter_index] -= step
197         jacobian[:, parameter_index] = (
198             weighted_residuals(shifted_plus) - weighted_residuals(shifted_minus)
199         ) / (2 * step)
200
201     covariance = np.linalg.pinv(jacobian.T @ jacobian)
202     population_transform = np.zeros((5, 5))
203     population_transform[0, 1:] = -1.0
204     population_transform[1:, 1:] = np.eye(4)
205     population_covariance = population_transform @ covariance @ population_transform.T
206     return np.sqrt(np.maximum(np.diag(population_covariance), 0.0))
207
208 def poisson_counts_for_negative_delays(t_fs, population_uncertainties, state_names,
209     maximum_delay_fs=50,):
210     """Calculate N = (Delta N)^2 up to a selected delay."""
211     negative_delays = t_fs <= maximum_delay_fs
212     poisson_counts = population_uncertainties[negative_delays] ** 2
213     return t_fs[negative_delays], poisson_counts
214
215 #---lets try it out---yipieyipieyipie
216 ref_file = Path("Daten/Analysis and Interpretation/Ref Spectrum/Reference_Data_Fig_4_4.
217     csv")
218 data_folder = Path("Daten/Experiment with Data Acquisition/messreihe")
219
220 delays = [-150, -100, -50, 0, 50, 100, 150, 200, 250, 300, 350, 400, 600, 800]
221 delay_names = ["min150 fs", "min100 fs", "min50 fs", "0 fs", "50 fs", "100 fs", "150 fs",
222     "200 fs", "250 fs", "300 fs", "350 fs", "400 fs", "600 fs", "800 fs"]
223
224 # deltaI matrix aus measurements
225 t_fs, energy_common, dI_mean, sigma = build_deltaI_matrix_from_folders(data_folder=
226     data_folder, delays=delays, delay_names=delay_names, load_spectrum=load_spectrum)
227 # load and interpolate all five reference spectra
228 energy_ref, reference_ref = load_reference_spectra(ref_file)
229 reference_spectra = np.column_stack([interp1d(energy_ref, reference_ref[:, state],
230     bounds_error=False, fill_value=np.nan)(energy_common)
231     for state in range(reference_ref.shape[1])])
232 valid = (np.isfinite(dI_mean).all(axis=0) & np.isfinite(sigma).all(axis=0)
233     & np.isfinite(reference_spectra).all(axis=1) & (sigma > 0).all(axis=0))
234
235 energy_fit = energy_common[valid]
236 dI_fit = dI_mean[:, valid]
237 sigma_fit = sigma[:, valid]
238 reference_fit = reference_spectra[valid]
239
240 amplitudes = []
241 populations = []
242 population_uncertainties = []
243 fitted_spectra = []
244
245 fit_until_50_fs = t_fs <= 50
246 (negative_amplitudes, negative_populations, negative_fitted_spectra,
247     negative_uncertainties, negative_result,
248 ) = fit_negative_delays_soft(dI_fit[fit_until_50_fs], sigma_fit[fit_until_50_fs],
249     reference_fit, regularization_strength=10.0)
250 negative_index = 0
251
252 for delay, difference, uncertainty in zip(t_fs, dI_fit, sigma_fit):
253     if delay <= 50:

```

```

247     amplitude = negative_amplitudes[negative_index]
248     population = negative_populations[negative_index]
249     fitted = negative_fitted_spectra[negative_index]
250     population_uncertainty = negative_uncertainties[negative_index]
251     negative_index += 1
252 else:
253     amplitude, population, fitted, result = fit_difference_spectrum(energy_fit,
254     difference, uncertainty, reference_fit)
255     population_uncertainty = estimate_population_uncertainties(result, difference,
256     uncertainty, reference_fit)
257     amplitudes.append(amplitude)
258     populations.append(population)
259     population_uncertainties.append(population_uncertainty)
260     fitted_spectra.append(fitted)
261     # print(f"{delay:>5.0f} fs: A = {amplitude:.5g}, populations = {population}")
262
263 amplitudes = np.array(amplitudes)
264 populations = np.array(populations)
265 population_uncertainties = np.array(population_uncertainties)
266 fitted_spectra = np.array(fitted_spectra)
267
268 state_names = ["singlet", "doublet", "triplet", "quartet", "quintet"]
269 state_colors = plt.rcParams["axes.prop_cycle"].by_key()["color"][:len(state_names)]
270 negative_delays, poisson_counts = poisson_counts_for_negative_delays(t_fs,
271     population_uncertainties, state_names, maximum_delay_fs=50)
272
273 # MAIN PLOT
274 plt.figure(figsize=(7, 4.5))
275 for index, state_name in enumerate(state_names):
276     plt.errorbar(
277         t_fs,
278         populations[:, index],
279         yerr=population_uncertainties[:, index],
280         fmt="o-",
281         capsize=3,
282         color=state_colors[index],
283         ecolor=(*to_rgba(state_colors[index])[:3], 0.5),
284         label=state_name,
285     )
286 plt.xlabel("Time delay [fs]")
287 plt.ylabel("Population $a_M$")
288 #plt.ylim(-0.02, 1.02)
289 plt.legend()
290 plt.tight_layout()
291 plt.savefig("Auswertung EXP21/fit/fit fixed neg delays/fit_try_to_fix_lol.png")
292 plt.show()
293
294 '''# PLOT COUNTS AT NEGATIVE DELAYS (über Fehlerbalken/Poisson Statistik)
295 number_of_negative_delays = len(negative_delays)
296 number_of_columns = 2
297 number_of_rows = int(np.ceil(number_of_negative_delays / number_of_columns))
298 fig, axes = plt.subplots(
299     number_of_rows,
300     number_of_columns,
301     figsize=(12, 3.5 * number_of_rows),
302     sharey=True,
303     squeeze=False,
304 )
305 axes = axes.ravel()
306
307 negative_indices = np.flatnonzero(t_fs <= 50)
308 for panel_index, (delay, counts, fit_index) in enumerate(zip(negative_delays,
309     poisson_counts, negative_indices)):
310     bars = axes[panel_index].bar(state_names, counts, color=state_colors)
311     axes[panel_index].set_title(
312         f"{delay:.0f} fs, A = {amplitudes[fit_index]:.5g}"
313     )

```

```

310     axes[panel_index].set_ylabel(r"$N = (\Delta N)^2$")
311     axes[panel_index].tick_params(axis="x", rotation=35)
312     axes[panel_index].grid(axis="y", alpha=0.25)
313     axes[panel_index].bar_label(bars, fmt="%.4g", padding=3)
314     axes[panel_index].set_ylim(0.0,0.9)
315
316 for axis in axes[number_of_negative_delays:]:
317     axis.set_visible(False)
318
319 fig.tight_layout()
320 fig.savefig("Auswertung EXP21/fit/fit fixed neg delays/
321             counts_negative_delays_per_delay_try_to_fix.png", dpi=200)
322 plt.show()'''
323
324 # Poisson-Statistik
325 # Beim Zählen von Photonen mit einem Detektor misst man
326 # eine Intensität über die Anzahl der gezählten Ereignisse N
327 # Für diese Verteilung gilt eine fundamentale Eigenschaft:
328 # Varianz = Mittelwert (      = N)
329 # Da der Fehlerbalken      N der Standardabweichung      entspricht, gilt:
330 # Delta N = \sqrt{N}
331 # Quadriert man den Fehlerbalken, erhält man die gemessene Anzahl (Intensität):
332 # (Delta N)^2 = N

```